جامعة الحسن الثاني بالدار البيضاء
كلية العلوم بن مسيك
FSBM
FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Université Hassan II de Casablanca

FACULTÉ DES SCIENCES BEN M'SIK

DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE

Infrastructure DevOps avec CI/CD, Kubernetes

PROJET FIN DE MODULE — MASTER DSBD & IA

Réalisé par :

Othman SALAHI

Ayman CHABBAKI

Malak HOUALI

Hiba DELLAJI

Encadré par :

Pr. Fouzia Benabbou

Année Universitaire 2025/2026

Résumé

Le domaine du développement logiciel moderne est confronté à des exigences croissantes en matière de *rapidité de livraison*, de *fiabilité* et de *scalabilité*. La mise en place d'une infrastructure **DevOps** automatisée est devenue une nécessité incontournable pour répondre à ces défis, en particulier dans des contextes d'applications intégrant de l'intelligence artificielle.

Ce projet propose la conception et l'implémentation d'une infrastructure DevOps complète sur **Microsoft Azure**, permettant de déployer l'application **AI Voice Interview** — une plateforme d'entretiens d'embauche automatisés par traitement vocal, développée avec *FastAPI*, *React/Vite* et *PostgreSQL*, et enrichie de modèles *Sentence-BERT* et *PyTorch*. L'infrastructure repose sur trois couches complémentaires : le provisionnement cloud avec **Terraform**, la configuration automatisée des serveurs avec **Ansible**, et l'orchestration des conteneurs avec **Kubernetes** (`kubeadm` + Flannel CNI).

Deux pipelines **Jenkins** ont été mis en place pour automatiser l'intégralité du cycle de vie : un pipeline d'infrastructure déclenché manuellement avec étape d'approbation, et un pipeline applicatif déclenché automatiquement via webhook **GitHub** à chaque `git push`, couvrant les phases de lint, test, build Docker, push vers Docker Hub et déploiement Kubernetes. Les bonnes pratiques de sécurité ont été appliquées à chaque couche (isolation réseau, gestion des secrets via Kubernetes Secrets, permissions SSH).

Abstract

Modern software development faces growing demands for *delivery speed*, *reliability* and *scalability*. Setting up an automated **DevOps** infrastructure has become a necessity to meet these challenges, especially in contexts involving artificial intelligence applications.

This project proposes the design and implementation of a complete DevOps infrastructure on **Microsoft Azure**, enabling the deployment of **AI Voice Interview** — an automated job interview platform powered by voice processing, built with *FastAPI*, *React/Vite* and *PostgreSQL*, and enhanced with *Sentence-BERT* and *PyTorch* models. The infrastructure relies on three complementary layers : cloud provisioning with **Terraform**, automated server configuration with **Ansible**, and container orchestration with **Kubernetes** (`kubeadm` + Flannel CNI).

Two **Jenkins** pipelines were implemented to automate the entire lifecycle : a manually triggered infrastructure pipeline with an approval gate, and an application pipeline automatically triggered via **GitHub** webhook on every `git push`, covering lint, testing, Docker build, Docker Hub push and Kubernetes deployment phases. Security best practices were applied at every layer (network isolation, secrets management via Kubernetes Secrets, SSH permissions).

Table des Matières

Résumé **2**

Abstract **4**

1 Introduction **15**
1.1 Contexte Général 16
1.2 Présentation du Projet 17
1.3 Objectifs du Projet 17
1.4 Stack Technologique 18

2 Architecture Globale **20**
2.1 Vue d'ensemble de l'Infrastructure 21
2.2 Architecture Cloud Azure 22
2.3 Architecture de l'Application 24
2.4 Flux CI/CD de Bout en Bout 25
2.5 Architecture Kubernetes 26

3 Présentation de l'Application **29**
3.1 Vue d'ensemble 30
3.2 Architecture Technique 30
3.2.1 Backend — FastAPI 30
3.2.2 Frontend — React / Vite 32
3.2.3 Base de Données — PostgreSQL 32
3.3 Modèles d'Intelligence Artificielle 33
3.3.1 Sentence-BERT 33
3.3.2 PyTorch 34
3.3.3 Pipeline d'Analyse d'une Réponse 35
3.4 Interface Utilisateur 36
3.4.1 Page d'Authentification 36
3.4.2 Tableau de Bord 37
3.4.3 Session d'Entretien 38
3.4.4 Page de Résultats 38

4 Conteneurisation avec Docker 39
4.1 Introduction à la Conteneurisation 40
4.2 Dockerfile du Backend (FastAPI) 40
4.3 Dockerfile du Frontend (React / Vite) 42
4.4 Configuration nginx (Proxy Inverse) 43
4.5 Docker Compose pour les Tests Locaux 44
4.6 Stratégie de Tagging des Images 46
4.7 Bonnes Pratiques Appliquées 46

5 Provisionnement de l'Infrastructure avec Terraform 48
5.1 Introduction à Terraform 49
5.2 Structure des Fichiers Terraform 49
5.3 Configuration du Provider Azure 50
5.4 Ressources Provisionnées 50
5.4.1 Machines Virtuelles 51
5.5 Variables et Sorties 52
5.5.1 Variables (`variables.tf`) 53
5.5.2 Sorties (`outputs.tf`) 53
5.6 Flux d'Exécution Terraform 53
5.7 Décisions de Conception 54

6 Configuration des Serveurs avec Ansible 55
6.1 Introduction à Ansible 56
6.2 Structure des Fichiers Ansible 56
6.3 Inventaire Dynamique 57
6.4 Playbook 1 : Installation de Docker 57
6.5 Playbook 2 : Configuration du Nœud Maître 58
6.6 Playbook 3 : Jonction des Workers 58
6.7 Playbook 4 : Récupération du Kubeconfig 58
6.8 Flux d'Exécution des Playbooks 58
6.9 Configuration Globale Ansible 59

7 Orchestration avec Kubernetes 61
7.1 Introduction à Kubernetes 62
7.2 Structure des Manifestes 62
7.3 Namespace 63
7.4 Gestion des Secrets 63
7.5 Persistance des Données PostgreSQL 63
7.5.1 PersistentVolume (`postgres-pv.yaml`) 64
7.5.2 PersistentVolumeClaim (`postgres-pvc.yaml`) 64
7.6 Déploiement de PostgreSQL 64
7.7 Déploiement du Backend 65
7.8 Déploiement du Frontend 65
7.9 Commandes de Vérification 66
7.10 Flux de Déploiement Kubernetes 67

8 Pipeline CI/CD avec Jenkins 69
8.1 Introduction 70
8.2 Configuration Globale du Pipeline 70
8.2.1 Variables d'Environnement 70
8.3 Étapes du Pipeline 71
8.3.1 Stage 1 : Checkout 71
8.3.2 Stage 2 : Check Docker Access 72
8.3.3 Stage 3 : Setup Python Environment 73
8.3.4 Stage 4 : Setup Frontend Environment 74
8.3.5 Stage 5 : Lint & Code Quality 75
8.3.6 Stage 6 : Run Frontend Tests 76
8.3.7 Stage 7 : Build Frontend 76
8.3.8 Stage 8 : Run Backend Tests 77
8.3.9 Stage 9 : Security Scan 78
8.3.10 Stage 10 : Build Docker Images 79
8.3.11 Stage 11 : Test Docker Images 80
8.3.12 Stage 12 : Push Docker Images 81
8.4 Bloc Post-Exécution 82

8.5 Flux Complet du Pipeline 82
8.6 Gestion des Credentials Jenkins 83
8.7 Points Techniques Remarquables 83

9 Conclusion 85
9.1 Synthèse du Travail Réalisé 86
9.2 Bilan des Objectifs Atteints 87
9.3 Difficultés Rencontrées 88
9.4 Perspectives d'Amélioration 89
9.5 Conclusion Générale 90

Liste des Figures

2.1 Vue d'ensemble de l'infrastructure DevOps en trois couches . . 21
2.2 Topologie réseau du cluster sur Microsoft Azure 24
2.3 Architecture interne de l'application AI Voice Interview 25
2.4 Flux de déploiement CI/CD de bout en bout 26
2.5 Ressources Kubernetes dans le namespace `ai-interview` . . . 28

3.1 Pipeline d'analyse d'une réponse vocale par le moteur IA 35
3.2 Page d'authentification — connexion et inscription 36
3.3 Page d'accueil — home page 37
3.4 Tableau de bord — historique des entretiens 37
3.5 Session d'entretien — question et enregistrement vocal 38
3.6 Page de résultats — score et recommandations IA 38

4.1 dockerfile de backend 42
4.2 dockerfile de frontend 43
4.3 la configuration de nginx 44
4.4 docker-composer.yml file 45
4.5 Flux de build et push des images Docker — stratégie 3 tags . . . 46

5.1 Flux d'exécution du pipeline Terraform dans Jenkins 54

6.1 Ordre d'exécution des playbooks Ansible et leurs cibles 59

7.1 Les nodes de cluster kubernetes 66
7.2 Les pods dans le cluster 67
7.3 Les services de cluster 67
7.4 Ordre d'application des manifestes Kubernetes lors du déploiement 68

8.1 stage checkout 72
8.2 stage docker access 73
8.3 stage setup python environment 74
8.4 stage setuup frontend environment 75
8.5 stage Lint and code quality 76
8.6 stage test et build de frontend 77

8.7 stage test de backend 78
8.8 stage security scan 79
8.9 stage build images de docker frontend et backend 80
8.10 stage test docker images 81
8.11 stage push images to docker hub 82
8.12 Flux complet du pipeline CI/CD Jenkins — 13 stages 83

Liste des Tableaux

1.1 Stack technologique du projet 19

2.1 Règles du Groupe de Sécurité Réseau (NSG) Azure 23
2.2 Ressources Kubernetes déployées dans le namespace `ai-interview` 27

3.1 Principaux endpoints de l'API REST (FastAPI) 31
3.2 Schéma simplifié de la base de données PostgreSQL 33
3.3 Caractéristiques du modèle Sentence-BERT utilisé 34

4.1 Comparaison image simple vs multi-stage pour le backend . . . 41
4.2 Stratégie de tagging des images Docker 46

5.1 Ressources Azure provisionnées par Terraform 51

7.1 Configuration de la persistance PostgreSQL 64

8.1 Variables d'environnement du pipeline Jenkins 71
8.2 Credentials Jenkins configurés pour le pipeline 83

9.1 Bilan des objectifs du projet 88

CHAPITRE 1

Introduction

1.1 Contexte Général

L'essor rapide du **cloud computing** et de l'**intelligence artificielle** a profondément transformé la manière dont les applications logicielles sont conçues, déployées et maintenues. Les équipes de développement modernes ne se contentent plus de livrer du code fonctionnel ; elles doivent garantir que ce code est déployé de façon *fiable*, *reproductible* et *continue*, dans des environnements cloud à grande échelle.

C'est dans ce contexte qu'émerge le mouvement **DevOps**, une culture et un ensemble de pratiques visant à rapprocher les équipes de développement (*Dev*) et d'exploitation (*Ops*). L'objectif est de briser les silos organisationnels et d'automatiser l'ensemble du cycle de vie logiciel, depuis l'intégration du code jusqu'à son déploiement en production.

Au cœur du DevOps se trouve la chaîne **CI/CD** (Intégration Continue / Déploiement Continu), qui permet de :

— Détecter les erreurs tôt grâce à des tests automatisés à chaque modification du code.
— Construire et packager l'application de manière cohérente via des conteneurs **Docker**.
— Déployer automatiquement sur un cluster **Kubernetes** sans intervention manuelle.
— Provisionner l'infrastructure cloud de façon déclarative avec **Terraform**.
— Configurer les serveurs de manière automatisée et idempotente avec **Ansible**.

Ce projet s'inscrit dans le cadre du Master **DSBD & IA** à la Faculté des Sciences Ben M'Sik, et a pour ambition de mettre en pratique l'ensemble de ces principes à travers le déploiement d'une application réelle d'intelligence artificielle : **AI Voice Interview**.

1.2 Présentation du Projet

AI Voice Interview est une plateforme web intelligente qui automatise les entretiens d'embauche par traitement vocal. L'application permet à des candidats d'interagir oralement avec un système d'IA, qui analyse leurs réponses en temps réel à l'aide de modèles de traitement du langage naturel basés sur *Sentence-BERT* et *PyTorch*, puis génère une évaluation structurée et un score de performance.

L'application est composée de trois composants principaux :

— **Backend** : une API REST développée avec *FastAPI* (Python), intégrant les modèles d'IA pour l'analyse sémantique des réponses vocales.

— **Frontend** : une interface utilisateur moderne développée avec *React / Vite*, permettant une interaction fluide et temps réel avec le système d'entretien.

— **Base de données** : *PostgreSQL*, utilisé pour stocker les profils des candidats, les sessions d'entretien et les résultats d'analyse.

Du point de vue DevOps, l'objectif est de mettre en place une infrastructure cloud complète sur **Microsoft Azure**, permettant de déployer, orchestrer et mettre à jour cette application de manière entièrement automatisée.

1.3 Objectifs du Projet

Les objectifs visés par ce projet sont les suivants :

1. **Conteneuriser l'application** avec Docker, en rédigeant des Dockerfiles optimisés (builds multi-étapes) pour le backend et le frontend.
2. **Provisionner l'infrastructure cloud** sur Microsoft Azure à l'aide de *Terraform*, en créant les machines virtuelles, le réseau virtuel, les groupes de sécurité et les adresses IP de manière déclarative et reproductible.
3. **Configurer automatiquement les serveurs** via des playbooks *Ansible*, incluant l'installation de Docker, la mise en place d'un cluster Kuber-

netes avec `kubeadm`, et le déploiement des composants réseau (*Flannel CNI*).

4. **Orchestrer l'application** sur un cluster Kubernetes, en définissant les manifestes YAML pour les Deployments, Services, PersistentVolumes et Secrets.
5. **Automatiser le cycle de déploiement** via deux pipelines Jenkins :
   — Un pipeline *infrastructure* pour le provisionnement et la configuration des serveurs Azure.
   — Un pipeline *applicatif* déclenché automatiquement à chaque `git push` via webhook GitHub.
6. **Appliquer les bonnes pratiques de sécurité** à chaque couche de l'infrastructure (isolation réseau, gestion des secrets Kubernetes, validation manuelle avant apply Terraform).

1.4 Stack Technologique

Le tableau 1.1 récapitule l'ensemble des outils et technologies utilisés dans ce projet, organisés par couche fonctionnelle.

TABLE 1.1 – Stack technologique du projet

Couche	Outil / Technologie	Rôle
Application	FastAPI + PyTorch	Backend REST avec moteur IA (NLP)
Application	React / Vite	Interface utilisateur SPA moderne
Base de données	PostgreSQL 15	Stockage relationnel persistant
Conteneurisation	Docker 29.3	Build, packaging et test des images
Orchestration	Kubernetes (kubeadm)	Gestion des conteneurs en cluster
Provisionnement	Terraform 1.14.7	Infrastructure as Code sur Azure
Configuration	Ansible 2.20.3	Configuration automatisée des serveurs
CI/CD	Jenkins (LTS)	Orchestration des pipelines
Cloud	Microsoft Azure	Fournisseur d'infrastructure cloud
Registre	Docker Hub	Dépôt des images de conteneurs

CHAPITRE 2

Architecture Globale

2.1 Vue d'ensemble de l'Infrastructure

L'infrastructure mise en place dans ce projet repose sur une organisation en **trois couches complémentaires et séquentielles**, chacune prenant en entrée les sorties de la couche précédente. Cette approche garantit une séparation claire des responsabilités et une automatisation de bout en bout.

1. **Couche 1 — Provisionnement** (*Terraform*) : création de toutes les ressources cloud sur Microsoft Azure (machines virtuelles, réseau, sécurité).
2. **Couche 2 — Configuration** (*Ansible*) : installation et configuration automatique de Docker et Kubernetes sur les machines provisionnées.
3. **Couche 3 — Orchestration** (*Kubernetes*) : déploiement et gestion des conteneurs de l'application (backend, frontend, base de données).

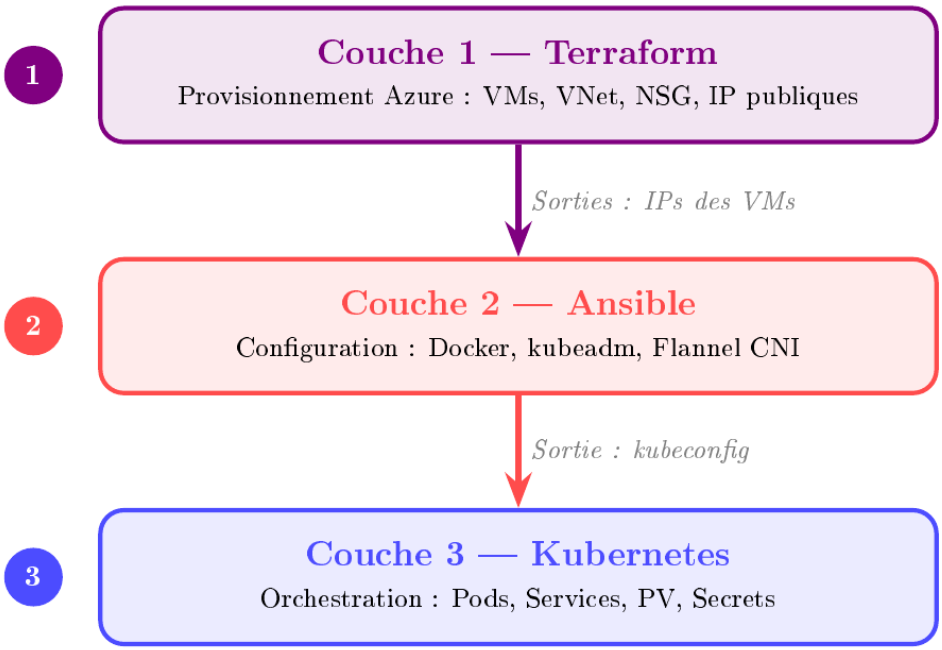

FIGURE 2.1 – Vue d'ensemble de l'infrastructure DevOps en trois couches

2.2 Architecture Cloud Azure

L'ensemble de l'infrastructure est hébergé sur **Microsoft Azure**. Les ressources sont organisées au sein d'un **groupe de ressources** dédié, contenant un réseau virtuel (*VNet*) avec un sous-réseau unique, protégé par un **groupe de sécurité réseau** (NSG) autorisant uniquement les ports nécessaires.

Le cluster est composé de **trois machines virtuelles** Ubuntu 22.04 LTS :

— **Serveur CI-CD** : héberge Jenkins, Terraform et Ansible. Possède une adresse IP publique pour recevoir les webhooks GitHub et lancer les pipelines.

— **Nœud Maître** (*Master Node*) : héberge le plan de contrôle Kubernetes (`kube-apiserver`, `etcd`, `controller-manager`). Possède une adresse IP publique pour l'accès à l'API Kubernetes (port 6443).

— **Nœud Worker** (*Worker Node*) : exécute les pods applicatifs (frontend, backend, PostgreSQL). Accessible uniquement via l'adresse IP privée, en passant par le nœud maître comme bastion SSH (*ProxyJump*).

Le tableau 2.1 récapitule les règles de sécurité réseau configurées sur le NSG.

TABLE 2.1 – Règles du Groupe de Sécurité Réseau (NSG) Azure

Port	Protocole	Direction	Usage
22	TCP	Entrant	Accès SSH aux machines
80	TCP	Entrant	Trafic HTTP
443	TCP	Entrant	Trafic HTTPS
6443	TCP	Entrant	API Kubernetes
30080	TCP	Entrant	Service NodePort Frontend
30800	TCP	Entrant	Service NodePort Backend
*	Tous	Interne	Communication entre nœuds du cluster

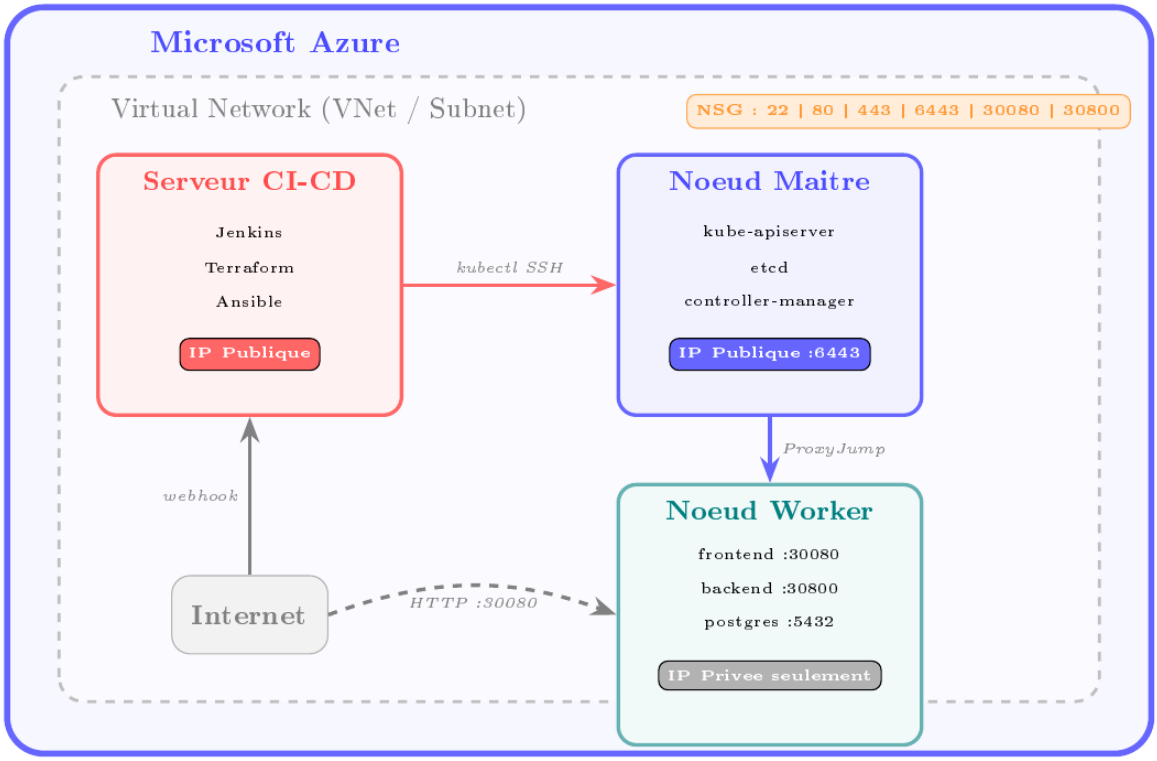

FIGURE 2.2 – Topologie réseau du cluster sur Microsoft Azure

2.3 Architecture de l'Application

L'application **AI Voice Interview** est composée de trois services principaux, déployés sous forme de conteneurs Docker orchestrés par Kubernetes :

— **Frontend** (React / Vite) : interface utilisateur exposée sur le port `30080`. Un serveur *nginx* intégré au conteneur sert les fichiers statiques et fait office de proxy inverse, routant les requêtes `/api/*` vers le service backend.

— **Backend** (FastAPI + PyTorch) : API REST exposée sur le port `30800`, gérant la logique métier, l'analyse vocale et les interactions avec la base de données.

— **Base de données** (PostgreSQL 15) : accessible uniquement en interne via le service `ClusterIP` sur le port `5432`. Les données sont persistées grâce à un *PersistentVolume*.

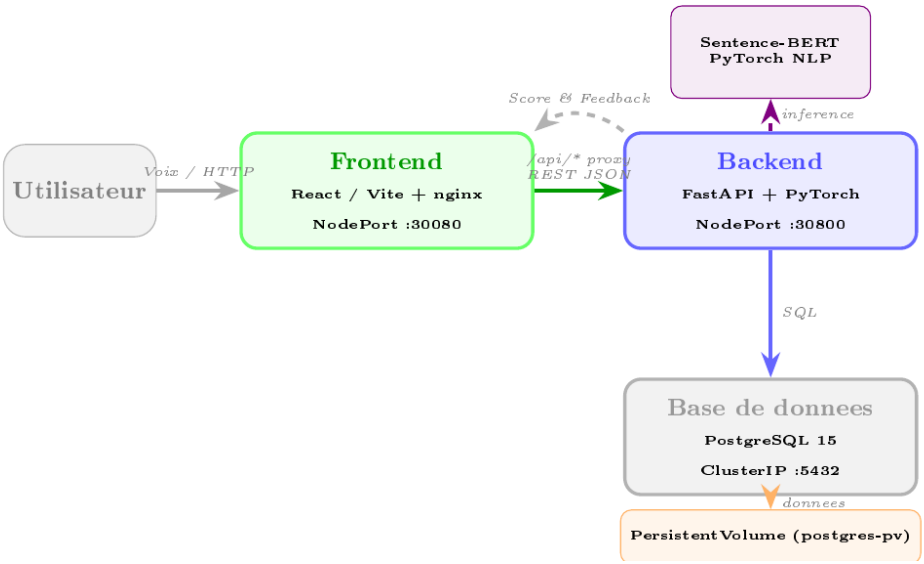

FIGURE 2.3 – Architecture interne de l'application AI Voice Interview

2.4 Flux CI/CD de Bout en Bout

Le flux de déploiement complet, depuis un `git push` du développeur jusqu'à la mise en production sur le cluster Kubernetes, implique les étapes suivantes :

1. Le développeur effectue un `git push` sur la branche `sadmad` du dépôt GitHub.
2. GitHub déclenche un **webhook** vers le serveur Jenkins.
3. Jenkins exécute le **pipeline applicatif** : lint → tests → build Docker → scan de sécurité → push des images vers Docker Hub.
4. Jenkins se connecte en SSH au nœud maître et applique les manifestes Kubernetes avec `kubectl apply`.
5. Kubernetes orchestre le déploiement des nouveaux pods sur le nœud worker.
6. L'application est disponible en **production** sur les ports NodePort.

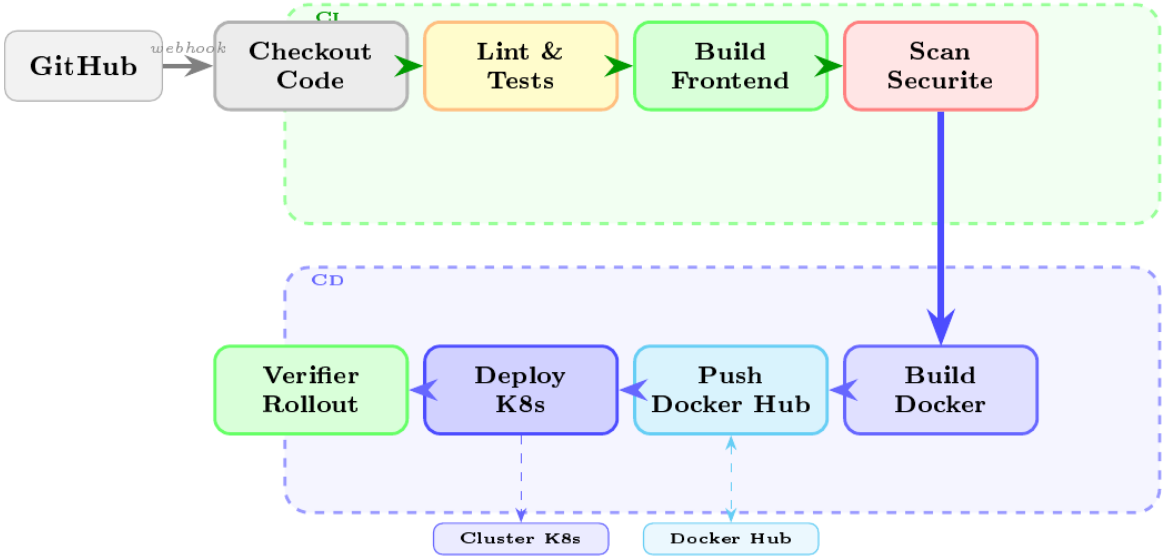

FIGURE 2.4 – Flux de déploiement CI/CD de bout en bout

2.5 Architecture Kubernetes

L'application est déployée dans un **namespace** dédié nommé `ai-interview`, assurant l'isolation logique des ressources au sein du cluster. Le tableau 2.2 résume les ressources Kubernetes déployées.

TABLE 2.2 – Ressources Kubernetes déployées dans le namespace `ai-interview`

Type	Nom	Description
Deployment	frontend	Pods nginx + React/Vite
Deployment	backend	Pods FastAPI + PyTorch
Deployment	postgres	Pod PostgreSQL 15
Service	frontend-svc	NodePort :30080 → pods frontend
Service	backend-svc	NodePort :30800 → pods backend
Service	postgres-svc	ClusterIP :5432 → pod postgres
PersistentVolume	postgres-pv	Volume physique sur le nœud worker
PersistentVolumeClaim	postgres-pvc	Réclamation du volume par le pod
Secret	db-credentials	Identifiants PostgreSQL chiffrés (base64)
Namespace	ai-interview	Isolation logique de tous les composants

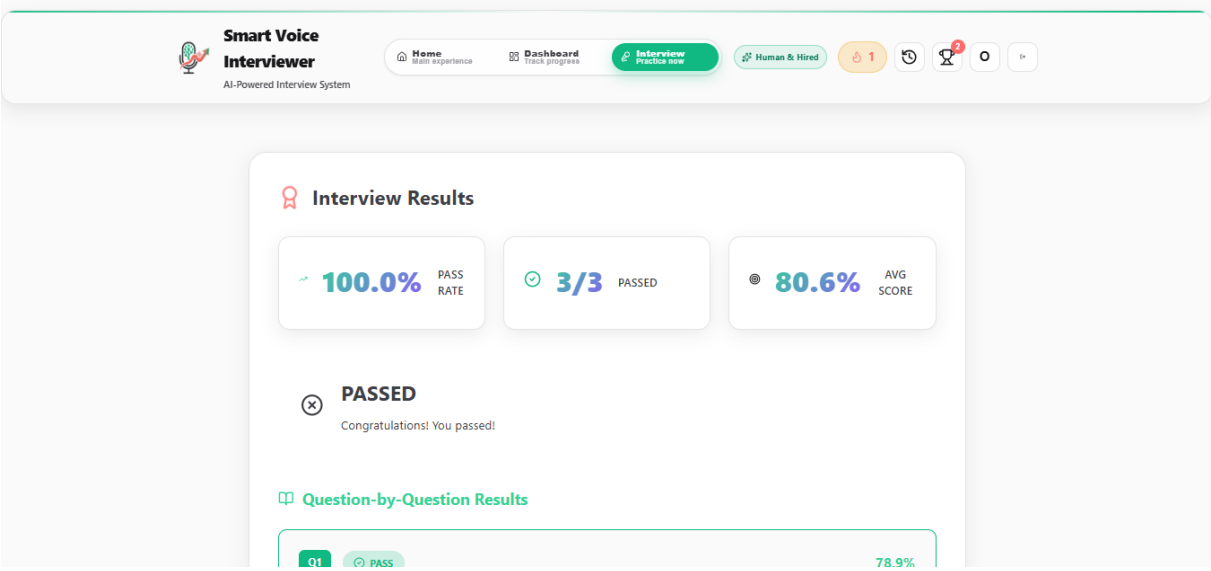

FIGURE 2.5 – Ressources Kubernetes dans le namespace `ai-interview`

CHAPITRE 3

Présentation de l'Application

3.1 Vue d'ensemble

AI Voice Interview est une plateforme web intelligente qui automatise les entretiens d'embauche à travers le traitement de la voix et l'analyse sémantique du langage naturel. Le système permet à un candidat d'interagir oralement avec un agent conversationnel IA, qui pose des questions, transcrit les réponses, les analyse en temps réel, puis génère une évaluation structurée incluant un score de performance et des recommandations.

L'application répond à plusieurs besoins concrets dans le domaine des ressources humaines :

— **Automatisation** des entretiens préliminaires sans intervention humaine.
— **Objectivité** de l'évaluation grâce à des modèles d'IA reproductibles.
— **Scalabilité** pour traiter un grand nombre de candidats simultanément.
— **Disponibilité** 24h/24, 7j/7, sans contraintes géographiques.

3.2 Architecture Technique

L'application suit une architecture **three-tier** (trois niveaux) classique, adaptée aux contraintes d'un déploiement Kubernetes :

1. **Couche présentation** : interface React/Vite servie par nginx.
2. **Couche métier** : API REST FastAPI intégrant les modèles IA.
3. **Couche données** : base de données relationnelle PostgreSQL.

3.2.1 Backend — FastAPI

Le backend est développé avec **FastAPI** (Python 3.11), un framework web moderne reconnu pour ses performances élevées grâce au support natif de l'*asynchronisme* (ASGI) et à la génération automatique de documentation interactive (Swagger UI / ReDoc).

Les composants principaux du backend sont :

— **Serveur ASGI** : *Uvicorn* gère les connexions HTTP et WebSocket.
— **ORM** : *SQLAlchemy* avec *Alembic* pour les migrations de schéma.
— **Authentification** : tokens *JWT* (JSON Web Tokens) pour sécuriser les endpoints.
— **Moteur IA** : intégration de *Sentence-BERT* et *PyTorch* pour l'analyse des réponses vocales.
— **Validation** : *Pydantic* pour la validation automatique des données entrantes et sortantes.

Le tableau 3.1 présente les principaux endpoints de l'API REST.

TABLE 3.1 – Principaux endpoints de l'API REST (FastAPI)

Méthode	Endpoint	Description
POST	/auth/register	Inscription d'un nouvel utilisateur
POST	/auth/login	Authentification et génération du token JWT
POST	/interview/start	Démarrer une nouvelle session d'entretien
POST	/interview/answer	Soumettre une réponse vocale (audio base64)
GET	/interview/{id}/res	Récupérer les résultats d'un entretien
GET	/interview/history	Historique des entretiens du candidat
GET	/health	Vérification de l'état du serveur (health check)

3.2.2 Frontend — React / Vite

Le frontend est une **Single-Page Application** (SPA) développée avec **React 18** et **Vite** comme bundler. Vite offre des temps de build très rapides grâce à l'utilisation des modules ES natifs et du pre-bundling avec *esbuild*.

Les principales technologies utilisées côté frontend sont :

— **React 18** : bibliothèque UI avec gestion d'état via hooks (`useState`, `useEffect`, `useContext`).
— **React Router** : navigation entre les pages sans rechargement.
— **Axios** : client HTTP pour les appels vers l'API backend.
— **Web Speech API** : capture audio du microphone du navigateur.
— **nginx** : serveur de fichiers statiques et proxy inverse en production.

En production, le fichier `nginx.conf` intégré dans le conteneur frontend configure le proxy inverse vers le backend :

```
location /api/ {
    proxy_pass http://backend-service:30800/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html;
}
```

3.2.3 Base de Données — PostgreSQL

PostgreSQL 15 est utilisé comme système de gestion de base de données relationnelle. Il stocke l'ensemble des données persistantes de l'application.

Le tableau 3.2 décrit les principales tables du schéma de données.

TABLE 3.2 – Schéma simplifié de la base de données PostgreSQL

Table	Colonnes principales	Description
`users`	id, email, password_hash, created_at	Profils des candidats et recruteurs
`interviews`	id, user_id, status, score, created_at	Sessions d'entretien avec score global
`questions`	id, interview_id, text, order	Questions posées lors de l'entretien
`answers`	id, question_id, audio_url, transcript, score	Réponses enregistrées et transcrites
`evaluations`	id, interview_id, feedback, recommendation	Évaluations générées par le moteur IA

3.3 Modèles d'Intelligence Artificielle

Le cœur de l'application repose sur deux technologies d'IA complémentaires : **Sentence-BERT** pour la compréhension sémantique du langage, et **PyTorch** comme framework d'inférence des modèles de deep learning.

3.3.1 Sentence-BERT

Sentence-BERT (SBERT) est une modification de l'architecture *BERT* (Bidirectional Encoder Representations from Transformers) entraînée spécifiquement pour produire des **embeddings de phrases** sémantiquement significatifs.

Contrairement à BERT classique qui encode des mots individuels, SBERT encode des phrases entières dans un espace vectoriel dense de dimension fixe (typiquement 384 ou 768 dimensions).

Dans le contexte de l'application, SBERT est utilisé pour :

— **Mesurer la similarité sémantique** entre la réponse du candidat et une réponse de référence attendue, via la similarité cosinus.
— **Évaluer la pertinence** de la réponse par rapport à la question posée.
— **Détecter les thèmes abordés** dans la réponse (compétences techniques, soft skills, expériences).

Le tableau 3.3 compare les principales caractéristiques du modèle utilisé.

TABLE 3.3 – Caractéristiques du modèle Sentence-BERT utilisé

Paramètre	Valeur
Modèle de base	`all-MiniLM-L6-v2`
Dimension embedding	384 dimensions
Métrique de similarité	Similarité cosinus
Taille du modèle	$\approx$ 80 MB
Vitesse d'inférence	$\approx$ 14 000 phrases/seconde (CPU)
Framework	`sentence-transformers` (HuggingFace)

3.3.2 PyTorch

PyTorch est le framework de deep learning open-source développé par *Meta AI*. Il sert de moteur d'inférence pour tous les modèles de l'application. Dans

ce projet, PyTorch est utilisé en mode **CPU uniquement** (sans GPU), ce qui est adapté aux machines virtuelles Azure de niveau gratuit.

Les rôles de PyTorch dans l'application sont :

— **Chargement des modèles pré-entraînés** (SBERT, classificateurs NLP) depuis les fichiers `.pt` ou depuis HuggingFace Hub.

— **Inférence** : calcul des embeddings et prédictions en temps réel.

— **Pipeline de traitement** : tokenisation, normalisation et post-traitement des résultats.

3.3.3 Pipeline d'Analyse d'une Réponse

Le traitement d'une réponse vocale suit un pipeline en quatre étapes successives, illustré dans la figure 3.1 :

1. **Transcription** : l'audio capturé par le navigateur est transcrit en texte via la *Web Speech API* côté client.
2. **Embedding** : SBERT encode la transcription en un vecteur de 384 dimensions.
3. **Scoring** : la similarité cosinus est calculée entre l'embedding de la réponse et celui de la réponse de référence.
4. **Évaluation** : un score normalisé entre 0 et 100 est calculé, accompagné d'un feedback textuel généré automatiquement.

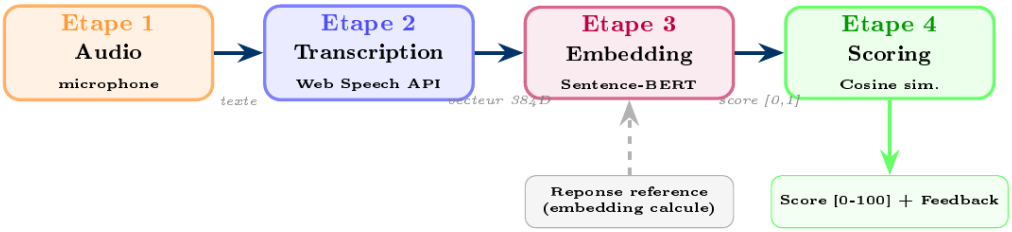

FIGURE 3.1 – Pipeline d'analyse d'une réponse vocale par le moteur IA

3.4 Interface Utilisateur

L'interface utilisateur de l'application est organisée en plusieurs écrans principaux, présentés ci-dessous. Les captures d'écran seront intégrées une fois l'application déployée.

3.4.1 Page d'Authentification

La page d'authentification permet aux candidats de s'inscrire ou de se connecter. Elle utilise un formulaire React avec validation côté client et gestion des erreurs.

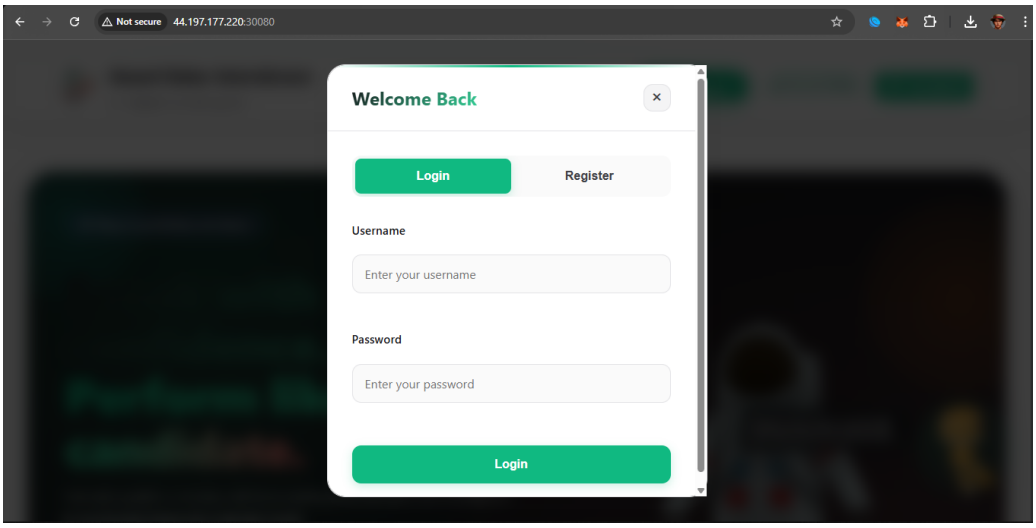

FIGURE 3.2 – Page d'authentification — connexion et inscription

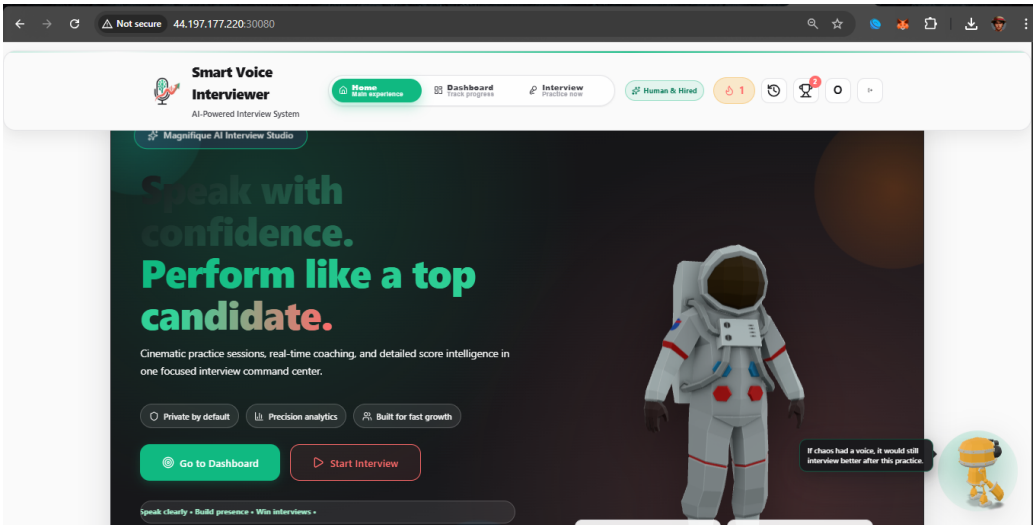

FIGURE 3.3 – Page d'accueil — home page

3.4.2 Tableau de Bord

Le tableau de bord affiche l'historique des entretiens du candidat, ses scores passés, et un bouton pour démarrer un nouvel entretien.

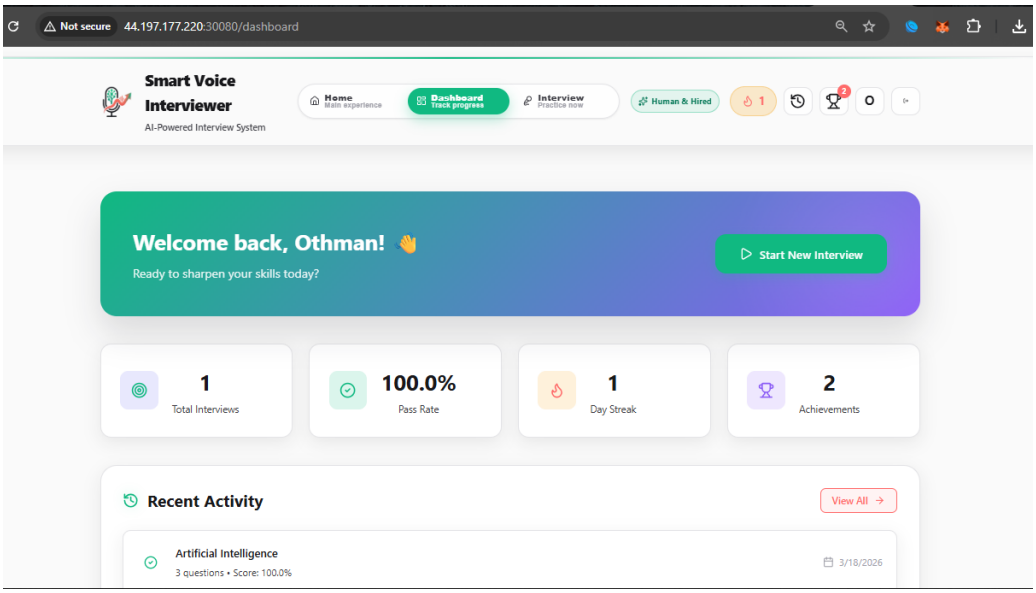

FIGURE 3.4 – Tableau de bord — historique des entretiens

3.4.3 Session d'Entretien

L'écran d'entretien affiche la question en cours, un indicateur d'enregistrement vocal, et un retour visuel en temps réel de la transcription.

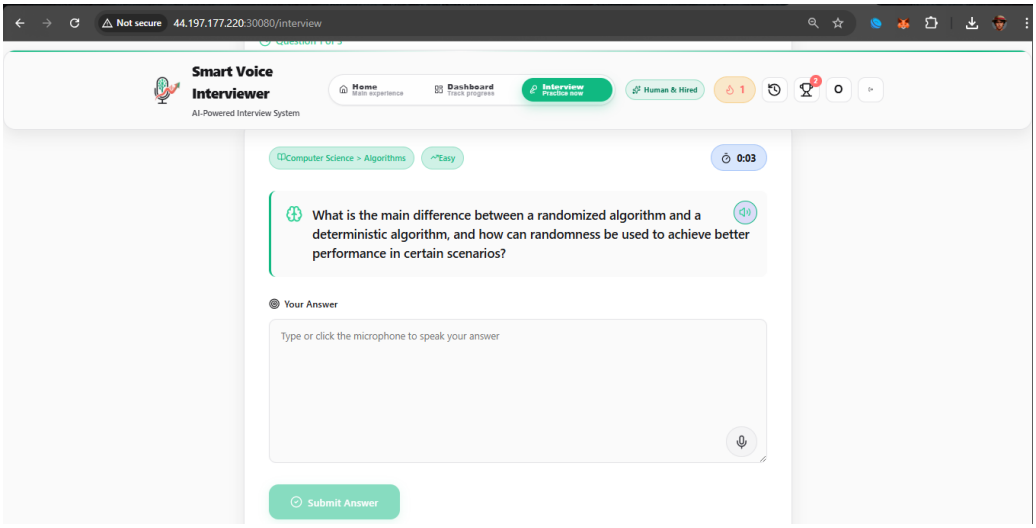

FIGURE 3.5 – Session d'entretien — question et enregistrement vocal

3.4.4 Page de Résultats

La page de résultats présente le score global de l'entretien, le détail par question, et les recommandations générées par le moteur IA.

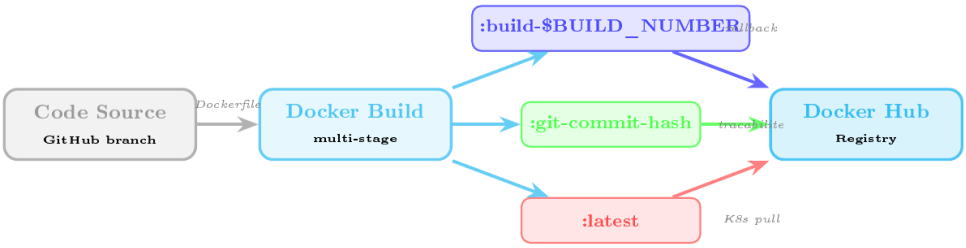

FIGURE 3.6 – Page de résultats — score et recommandations IA

CHAPITRE 4

Conteneurisation avec Docker

4.1 Introduction à la Conteneurisation

La **conteneurisation** est une technique de virtualisation légère qui permet d'empaqueter une application avec toutes ses dépendances (bibliothèques, variables d'environnement, fichiers de configuration) dans une unité isolée et portable appelée **conteneur**. Contrairement aux machines virtuelles, les conteneurs partagent le noyau du système d'exploitation hôte, ce qui les rend beaucoup plus légers et rapides à démarrer.

Dans ce projet, **Docker** est utilisé pour :

— **Garantir la reproductibilité** : le même conteneur s'exécute identiquement en développement, en test et en production.

— **Isoler les services** : backend, frontend et base de données fonctionnent dans des environnements cloisonnés.

— **Simplifier le déploiement** : Kubernetes orchestre directement les images Docker sans configuration manuelle des serveurs.

— **Versionner l'infrastructure** : chaque image est taguée avec le numéro de build et le hash du commit Git.

4.2 Dockerfile du Backend (FastAPI)

Le backend utilise un **build multi-étapes** (*multi-stage build*) pour minimiser la taille de l'image finale. Cette approche sépare l'environnement de compilation de l'environnement d'exécution, éliminant les outils de build inutiles en production.

TABLE 4.1 – Comparaison image simple vs multi-stage pour le backend

Type	Image de base	Taille estimée	Production
Image simple	python :3.11	$\sim$ 1.2 GB	Non recommandé
Multi-stage (slim)	python :3.11-slim	$\sim$ 380 MB	Recommandé
Multi-stage (final)	python :3.11-slim	$\sim$ 280 MB	Optimal

Voici le `Dockerfile` du backend :

```
# ── Etape 1 : Build ──────────────────────────────────────
FROM python:3.11-slim AS builder

WORKDIR /app

# Installer les dependances systeme necessaires au build
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc g++ && rm -rf /var/lib/apt/lists/*

# Copier et installer les dependances Python
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ── Etape 2 : Production ─────────────────────────────────
FROM python:3.11-slim AS production

WORKDIR /app

# Copier uniquement les packages installes depuis l'etape builder
COPY --from=builder /install /usr/local

# Copier le code source de l'application
COPY . .

# Exposer le port de l'API
EXPOSE 8000

# Lancer le serveur Uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

FIGURE 4.1 – dockerfile de backend

4.3 Dockerfile du Frontend (React / Vite)

Le frontend utilise également un build multi-étapes : la première étape compile l'application React avec *Node.js*, et la deuxième étape sert les fichiers statiques générés via **nginx**, qui fait également office de proxy inverse vers le backend.

```
# ── Etape 1 : Build React / Vite ──────────────────────────
FROM node:20-alpine AS builder

WORKDIR /app

# Installer les dependances npm
COPY package*.json ./
RUN npm ci --only=production

# Copier les sources et compiler
COPY . .
RUN npm run build

# ── Etape 2 : Serveur nginx ───────────────────────────────
FROM nginx:alpine AS production

# Copier les fichiers statiques compiles
COPY --from=builder /app/dist /usr/share/nginx/html

# Copier la configuration nginx (proxy vers backend)
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

FIGURE 4.2 – dockerfile de frontend

4.4 Configuration nginx (Proxy Inverse)

Le fichier `nginx.conf` est au cœur de la communication entre le frontend et le backend. Il configure nginx pour servir les fichiers statiques React et router automatiquement les requêtes `/api/*` vers le service backend Kubernetes, évitant ainsi les problèmes de CORS (*Cross-Origin Resource Sharing*).

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    # ── Proxy all backend API routes ──────────
    location ~ ^/(register|login|me|categories|statistics|interview|profile|health|docs|redoc|openapi.jso
n) {
        proxy_pass http://backend.ai-interview.svc.cluster.local:8000;
        proxy_http_version 1.1;
        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header Authorization       $http_authorization;
        proxy_pass_header Authorization;
        proxy_read_timeout    120s;
        proxy_connect_timeout 120s;
    }

    # ── React frontend ────────────────────────
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

FIGURE 4.3 – la configuration de nginx

4.5 Docker Compose pour les Tests Locaux

Un fichier `docker-compose.yml` est utilisé dans le pipeline Jenkins pour démarrer un environnement de test complet avant de pousser les images vers Docker Hub. Il orchestre les trois services ensemble et valide le *health check* de l'API.

```yaml
version: '3.8'

services:
  # PostgreSQL Database
  db:
    image: postgres:15-alpine
    container_name: ai-interview-db
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-interview_user}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-interview_password}
      POSTGRES_DB: ${POSTGRES_DB:-interview_db}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-interview_user} -d ${POSTGRES_DB:-interview_db}"
]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - ai-interview-network

  # Backend API
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: ai-interview-backend
    environment:
      # Database (uses .env variables)
      DATABASE_URL: ${DATABASE_URL:-postgresql://interview_user:interview_password@db:5432/interview_db}
      DB_POOL_SIZE: ${DB_POOL_SIZE:-10}

      # Security
      SECRET_KEY: ${SECRET_KEY:-your-secret-key-change-in-production}
      ACCESS_TOKEN_EXPIRE_MINUTES: ${ACCESS_TOKEN_EXPIRE_MINUTES:-10080}

      # Interview settings
      SIMILARITY_THRESHOLD: ${SIMILARITY_THRESHOLD:-0.6}
      NUM_QUESTIONS: ${NUM_QUESTIONS:-3}

      # CORS
      ALLOWED_ORIGINS: ${ALLOWED_ORIGINS:-http://localhost:5173,http://localhost:3000}

      # Server
      PORT: ${PORT:-8000}
      WORKERS: ${WORKERS:-1}
      LOG_LEVEL: ${LOG_LEVEL:-info}
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
      - model_cache:/app/.cache
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import requests; requests.get('http://localhost:8000/health')"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    networks:
      - ai-interview-network
    restart: unless-stopped

  # Frontend (Optional - uncomment if you want to run frontend in Docker)
    frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: ai-interview-frontend
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - ai-interview-network
    restart: unless-stopped

volumes:
  postgres_data:
    driver: local
  model_cache:
    driver: local

networks:
  ai-interview-network:
    driver: bridge
```

FIGURE 4.4 – docker-composer.yml file

4.6 Stratégie de Tagging des Images

Chaque build Jenkins produit **trois tags** pour chaque image (backend et frontend), offrant à la fois la traçabilité et la flexibilité de rollback.

TABLE 4.2 – Stratégie de tagging des images Docker

Tag	Exemple	Utilité
`:build-N`	`:build-42`	Numéro de build Jenkins — rollback précis
`:commit-hash`	`:a3f8b2c`	Hash Git court — lien direct vers le code source
`:latest`	`:latest`	Toujours la dernière version — utilisé par Kubernetes

La figure 4.5 illustre le flux complet de build et push des images Docker dans le pipeline Jenkins.

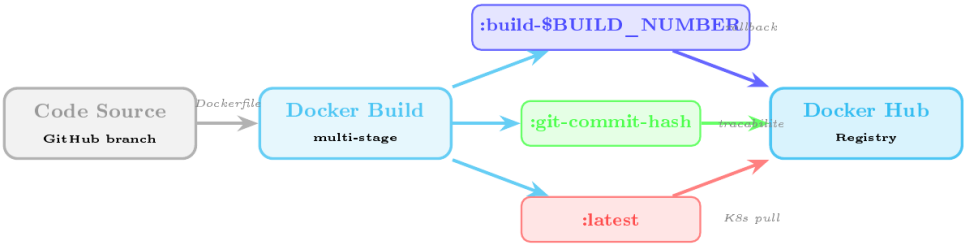

FIGURE 4.5 – Flux de build et push des images Docker — stratégie 3 tags

4.7 Bonnes Pratiques Appliquées

Les bonnes pratiques suivantes ont été appliquées dans la conception des Dockerfiles :

- **Build multi-étapes** : séparation stricte entre environnement de build et environnement d'exécution pour réduire la surface d'attaque et la taille des images.
- **Images de base légères** : utilisation de `python:3.11-slim` et `nginx:alpine` au lieu des images complètes.
- **Cache des layers** : les fichiers `requirements.txt` et `package.json` sont copiés avant le code source pour exploiter le cache Docker et accélérer les builds.
- **Utilisateur non-root** : les conteneurs de production s'exécutent avec un utilisateur sans privilèges pour limiter les risques de sécurité.
- **Health checks** : chaque service définit un health check permettant à Kubernetes et Docker Compose de vérifier l'état de santé du conteneur.
- **Variables d'environnement** : aucun secret n'est codé en dur dans les Dockerfiles ; les credentials sont injectés via les Secrets Kubernetes ou les variables Jenkins.

CHAPITRE 5

Provisionnement de l'Infrastructure avec Terraform

5.1 Introduction à Terraform

Terraform est un outil d'*Infrastructure as Code* (IaC) développé par HashiCorp, permettant de définir, provisionner et gérer des ressources cloud de manière **déclarative** à travers des fichiers de configuration HCL (*HashiCorp Configuration Language*). Dans ce projet, Terraform est utilisé pour créer l'intégralité des ressources Microsoft Azure à partir de zéro, de manière reproductible et versionnée.

Les avantages clés de Terraform dans ce projet sont :

— **Déclaratif** : on décrit l'*état souhaité* de l'infrastructure, Terraform se charge de l'atteindre.

— **Idempotent** : appliquer la même configuration plusieurs fois produit toujours le même résultat.

— **Plan avant Apply** : la commande `terraform plan` permet de prévisualiser les changements avant de les appliquer.

— **State management** : Terraform maintient un fichier d'état (`terraform.tfstate`) décrivant les ressources réellement créées.

5.2 Structure des Fichiers Terraform

Le répertoire Terraform du projet est organisé comme suit :

```
terraform/
|-- main.tf          # Ressources Azure principales
|-- variables.tf     # Déclaration des variables
|-- outputs.tf       # Sorties (IPs, noms)
|-- providers.tf     # Configuration du provider azurerm
|-- terraform.tfvars # Valeurs des variables (gitignored)
```

5.3 Configuration du Provider Azure

Le fichier `providers.tf` configure le provider `azurerm` avec les credentials du *Service Principal* Azure, stockés dans les variables Jenkins Credentials (jamais en clair dans le code) :

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
  required_version = ">= 1.14.7"
}

provider "azurerm" {
  features {}

  subscription_id = var.subscription_id
  client_id       = var.client_id
  client_secret   = var.client_secret
  tenant_id       = var.tenant_id
}
```

5.4 Ressources Provisionnées

Le fichier `main.tf` définit toutes les ressources Azure nécessaires au cluster. Le tableau 5.1 récapitule l'ensemble des ressources créées.

TABLE 5.1 – Ressources Azure provisionnées par Terraform

Ressource Azure	Quantité	Description
Resource Group	1	Conteneur logique pour toutes les ressources
Virtual Network (VNet)	1	Réseau isolé avec plage IP `10.0.0.0/16`
Subnet	1	Sous-réseau `10.0.1.0/24`
Network Security Group	1	Règles de pare-feu (ports 22, 80, 443, 6443)
Public IP (statique)	2	Pour le serveur CI-CD et le nœud maître
Network Interface (NIC)	3	Une par machine virtuelle
Virtual Machine (Ubuntu)	3	CI-CD, Master Node, Worker Node
OS Disk	3	Disques de 30 GB par VM

5.4.1 Machines Virtuelles

Les trois machines virtuelles sont provisionnées avec Ubuntu 22.04 LTS. La configuration ci-dessous montre le nœud maître à titre d'exemple :

```
resource "azurerm_linux_virtual_machine" "master" {
  name                = "master-node"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  size                = "Standard_B2s"
  admin_username      = "azureuser"

  admin_ssh_key {
    username   = "azureuser"
    public_key = file(var.ssh_public_key_path)
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
    disk_size_gb         = 30
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  network_interface_ids = [azurerm_network_interface.master.id]
}
```

5.5 Variables et Sorties

5.5.1 Variables (variables.tf)

```
variable "location"            { default = "West Europe" }
variable "resource_group_name" { default = "ai-interview-rg" }
variable "subscription_id"     { sensitive = true }
variable "client_id"           { sensitive = true }
variable "client_secret"       { sensitive = true }
variable "tenant_id"           { sensitive = true }
variable "ssh_public_key_path" { default = "~/.ssh/azure_key.pub" }
```

5.5.2 Sorties (outputs.tf)

Les sorties Terraform sont directement consommées par le pipeline Jenkins pour générer l'inventaire Ansible dynamiquement :

```
output "master_public_ip" {
  value       = azurerm_public_ip.master.ip_address
  description = "Adresse IP publique du noeud maitre"
}

output "worker_private_ip" {
  value       = azurerm_network_interface.worker.private_ip_address
  description = "Adresse IP privee du noeud worker"
}

output "cicd_public_ip" {
  value       = azurerm_public_ip.cicd.ip_address
  description = "Adresse IP publique du serveur CI-CD"
}
```

5.6 Flux d'Exécution Terraform

La figure 5.1 illustre le flux d'exécution du pipeline Terraform dans Jenkins, incluant l'étape d'approbation manuelle obligatoire.

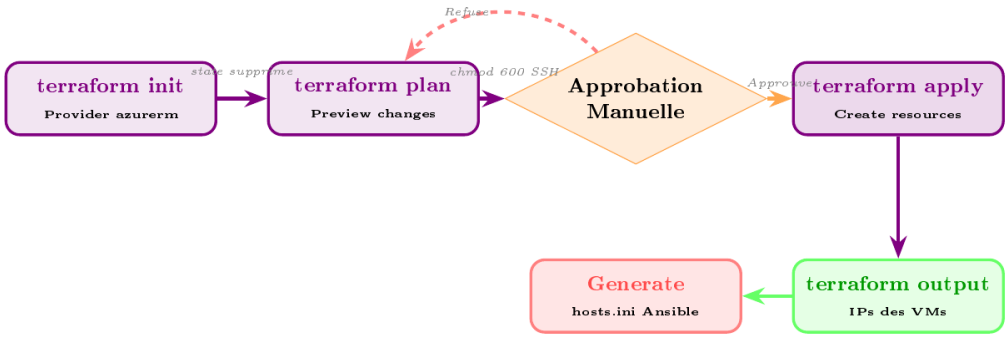

FIGURE 5.1 – Flux d'exécution du pipeline Terraform dans Jenkins

5.7 Décisions de Conception

Plusieurs décisions importantes ont été prises dans la configuration Terraform :

— **Suppression du state à chaque run** : le fichier `terraform.tfstate` est supprimé au début de chaque exécution du pipeline infrastructure. Puisque l'infrastructure est entièrement recréée à chaque run, cette approche évite les conflits avec des ressources supprimées manuellement entre deux exécutions.

— **Workers sans IP publique** : les nœuds workers n'ont aucune adresse IP publique. Ils sont accessibles uniquement via SSH ProxyJump à travers le nœud maître, réduisant significativement la surface d'attaque.

— **Taille des VMs** : les machines de type `Standard_B2s` (2 vCPU, 4 GB RAM) ont été choisies pour leur équilibre entre performance et coût, compatibles avec les niveaux gratuits Azure.

— **Credentials sensibles** : les variables `client_secret`, `subscription_id` et `tenant_id` sont marquées `sensitive = true` et injectées via Jenkins Credentials, jamais stockées dans le dépôt Git.

CHAPITRE 6

Configuration des Serveurs avec Ansible

6.1 Introduction à Ansible

Ansible est un outil de gestion de configuration et d'automatisation *sans agent* développé par Red Hat. Il communique avec les serveurs distants via **SSH** et exécute des tâches définies dans des fichiers YAML appelés **playbooks**. Son caractère *idempotent* garantit que l'exécution répétée d'un playbook produit toujours le même résultat final, sans effets de bord.

Dans ce projet, Ansible intervient en **deuxième couche** après Terraform, pour configurer automatiquement les machines virtuelles Azure fraîchement provisionnées. Les tâches principales réalisées sont :

— Installation de **Docker CE** sur tous les nœuds.
— Initialisation du **cluster Kubernetes** via `kubeadm` sur le nœud maître.
— Jonction des **nœuds workers** au cluster via le token généré.
— Installation du plugin réseau **Flannel CNI** pour la communication entre pods.
— Récupération du fichier **kubeconfig** vers le serveur Jenkins.

6.2 Structure des Fichiers Ansible

Le répertoire Ansible du projet est organisé comme suit :

```
ansible/
|-- hosts.ini              # Inventaire (généré dynamiquement)
|-- ansible.cfg            # Configuration globale Ansible
|-- install-docker.yml     # Playbook 1 : installation Docker
|-- setup-master.yml       # Playbook 2 : initialisation K8s mas
|-- setup-workers.yml      # Playbook 3 : jonction des workers
|-- get-kubeconfig.yml     # Playbook 4 : récupération kubeconfi
```

6.3 Inventaire Dynamique

L'inventaire Ansible (`hosts.ini`) est **généré automatiquement** par le pipeline Jenkins à partir des sorties Terraform. Cette approche élimine toute configuration manuelle des adresses IP et garantit la cohérence entre l'infrastructure provisionnée et la configuration appliquée.

Les workers n'ayant pas d'adresse IP publique, ils sont configurés avec `ansible_ssh_common_args` pour utiliser le nœud maître comme **bastion SSH** via `ProxyJump` :

```
# hosts.ini - généré automatiquement depuis les outputs Terraform
[cicd]
cicd-server ansible_host=<CICD_PUBLIC_IP> ansible_user=azureuser

[master]
master ansible_host=<MASTER_PUBLIC_IP> ansible_user=azureuser

[workers]
worker-01 ansible_host=<WORKER_PRIVATE_IP>
          ansible_ssh_common_args='-o StrictHostKeyChecking=no
          -o ProxyJump=azureuser@<MASTER_PUBLIC_IP>'

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=/home/jenkins/.ssh/azure_key
ansible_python_interpreter=/usr/bin/python3
```

6.4 Playbook 1 : Installation de Docker

Le playbook `install-docker.yml` installe Docker CE sur tous les nœuds du cluster en suivant la procédure officielle Docker pour Ubuntu.

6.5 Playbook 2 : Configuration du Nœud Maître

Le playbook `setup-master.yml` prépare le nœud maître Kubernetes en effectuant les étapes suivantes : désactivation du swap, chargement des modules kernel nécessaires, initialisation du cluster avec `kubeadm`, et installation du plugin réseau Flannel.

6.6 Playbook 3 : Jonction des Workers

Le playbook `setup-workers.yml` connecte les nœuds workers au cluster en exécutant la commande de jonction générée par le maître. La connexion SSH passe par le maître comme bastion via ProxyJump.

6.7 Playbook 4 : Récupération du Kubeconfig

Le playbook `get-kubeconfig.yml` copie le fichier `admin.conf` depuis le nœud maître vers le serveur Jenkins, permettant à `kubectl` de gérer le cluster à distance depuis les pipelines CI/CD.

6.8 Flux d'Exécution des Playbooks

La figure 6.1 illustre l'ordre d'exécution des quatre playbooks Ansible et les dépendances entre eux.

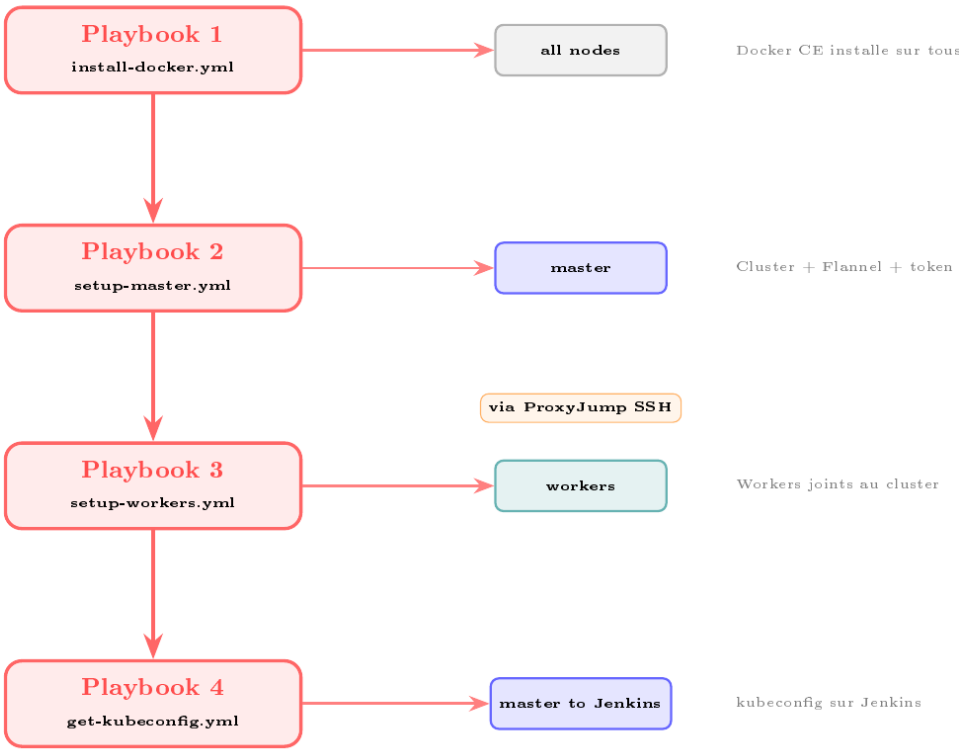

FIGURE 6.1 – Ordre d'exécution des playbooks Ansible et leurs cibles

6.9 Configuration Globale Ansible

Le fichier `ansible.cfg` définit les paramètres globaux pour éviter les erreurs de connexion SSH et optimiser les performances :

```
[defaults]
host_key_checking   = False
remote_user         = azureuser
private_key_file    = /home/jenkins/.ssh/azure_key
interpreter_python  = /usr/bin/python3
timeout             = 60
forks               = 5

[ssh_connection]
ssh_args            = -o ControlMaster=auto
                      -o ControlPersist=60s
                      -o StrictHostKeyChecking=no
pipelining          = True
```

CHAPITRE 7

Orchestration avec Kubernetes

7.1 Introduction à Kubernetes

Kubernetes (K8s) est la plateforme d'orchestration de conteneurs open-source de référence, développée initialement par Google. Elle automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées sur un cluster de machines.

Dans ce projet, Kubernetes est utilisé pour orchestrer les trois services de l'application **AI Voice Interview** sur le cluster Azure provisionné par Terraform et configuré par Ansible. Le cluster est initialisé avec `kubeadm` et utilise **Flannel CNI** pour la communication réseau entre les pods.

Les principaux avantages apportés par Kubernetes dans ce projet sont :

— **Auto-healing** : redémarrage automatique des pods en cas de défaillance.
— **Mise à l'échelle** : possibilité d'augmenter le nombre de réplicas à la demande.
— **Rolling updates** : mise à jour progressive sans interruption de service.
— **Gestion des secrets** : stockage sécurisé des credentials via les Kubernetes Secrets.
— **Persistance des données** : maintien des données PostgreSQL même après redémarrage des pods.

7.2 Structure des Manifestes

Les manifestes Kubernetes sont organisés dans le répertoire `k8s/` du projet. Chaque fichier YAML définit une ou plusieurs ressources Kubernetes :

```
k8s/
|-- namespace.yaml        # Namespace ai-interview
|-- secret.yaml           # Secrets (credentials DB)
|-- postgres-pv.yaml      # PersistentVolume PostgreSQL
|-- postgres-pvc.yaml     # PersistentVolumeClaim
|-- postgres-deploy.yaml  # Deployment + Service PostgreSQL
|-- backend-deploy.yaml   # Deployment + Service Backend
|-- frontend-deploy.yaml  # Deployment + Service Frontend
```

7.3 Namespace

Le fichier `namespace.yaml` crée un namespace dédié `ai-interview` qui assure l'isolation logique de toutes les ressources de l'application au sein du cluster. Cela permet de séparer les ressources du projet de celles du système Kubernetes (`kube-system`) et de tout autre déploiement futur.

7.4 Gestion des Secrets

Le fichier `secret.yaml` définit un **Kubernetes Secret** de type `Opaque` contenant les credentials sensibles de l'application encodés en *base64* :

— Le mot de passe de la base de données PostgreSQL.

— La clé secrète de l'application (JWT secret key).

— L'URL de connexion complète à la base de données.

Ces secrets sont injectés dans les pods backend et postgres sous forme de variables d'environnement, sans jamais apparaître en clair dans le code source ou les logs.

7.5 Persistance des Données PostgreSQL

La persistance des données PostgreSQL est assurée par deux ressources complémentaires.

7.5.1 PersistentVolume (`postgres-pv.yaml`)

Le fichier `postgres-pv.yaml` définit un volume physique de type `hostPath` de **5 GB** sur le nœud worker. Ce volume existe indépendamment des pods et survit à leurs redémarrages. Il est configuré avec le mode d'accès `ReadWriteOnce` (lecture/écriture par un seul nœud à la fois).

7.5.2 PersistentVolumeClaim (`postgres-pvc.yaml`)

Le fichier `postgres-pvc.yaml` définit la réclamation du volume par le pod PostgreSQL. Il demande **5 GB** de stockage avec le mode d'accès `ReadWriteOnce`. Kubernetes fait correspondre automatiquement cette réclamation avec le PersistentVolume disponible.

TABLE 7.1 – Configuration de la persistance PostgreSQL

Paramètre	PersistentVolu	PersistentVolu
Capacité	5 Gi	5 Gi
Mode d'accès	ReadWriteOnce	ReadWriteOnce
Type	hostPath	—
Chemin hôte	`/mnt/postgres`	—
Politique de récup.	Retain	—

7.6 Déploiement de PostgreSQL

Le fichier `postgres-deploy.yaml` contient deux ressources Kubernetes :

— **Deployment** : définit un pod PostgreSQL 15 avec une réplique, monté sur le PVC pour la persistance. Les variables d'environnement (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`) sont injectées depuis le Secret Kubernetes. Un *liveness probe* vérifie périodiquement la santé du pod via

`pg_isready`.

— **Service ClusterIP** : expose PostgreSQL sur le port `5432` uniquement à l'intérieur du cluster (pas d'accès externe), accessible par le backend via le nom DNS du service.

7.7 Déploiement du Backend

Le fichier `backend-deploy.yaml` contient :

— **Deployment** : définit un pod FastAPI avec une réplique, utilisant l'image Docker pushée sur Docker Hub. Les variables d'environnement (`DATABASE_URL`, `SECRET_KEY`) sont injectées depuis le Secret. La directive `imagePullPolicy: Always` garantit que Kubernetes télécharge toujours la dernière image `:latest` à chaque redéploiement. Un *readiness probe* sur l'endpoint `/health` vérifie que le pod est prêt avant de recevoir du trafic.

— **Service NodePort** : expose le backend sur le port `30800` de tous les nœuds du cluster, permettant l'accès depuis l'extérieur.

7.8 Déploiement du Frontend

Le fichier `frontend-deploy.yaml` contient :

— **Deployment** : définit un pod nginx + React/Vite avec une réplique, utilisant l'image Docker du frontend. nginx sert les fichiers statiques compilés et fait office de proxy inverse vers le service backend via la configuration `nginx.conf` intégrée dans l'image.

— **Service NodePort** : expose le frontend sur le port `30080` de tous les nœuds du cluster, constituant le point d'entrée principal de l'application pour les utilisateurs.

7.9 Commandes de Vérification

Après déploiement, les commandes suivantes permettent de vérifier l'état du cluster et des ressources :

```
# Vérifier l'état des noeuds
kubectl get nodes

# Lister tous les pods du namespace
kubectl get pods -n ai-interview

# Lister les services exposés
kubectl get svc -n ai-interview

# Vérifier la persistance des données
kubectl get pv,pvc -n ai-interview

# Logs du pod backend (debugging)
kubectl logs -n ai-interview deployment/backend

# Décrire un pod en détail
kubectl describe pod -n ai-interview <nom-du-pod>
```

Le résultat attendu après un déploiement réussi est le suivant :

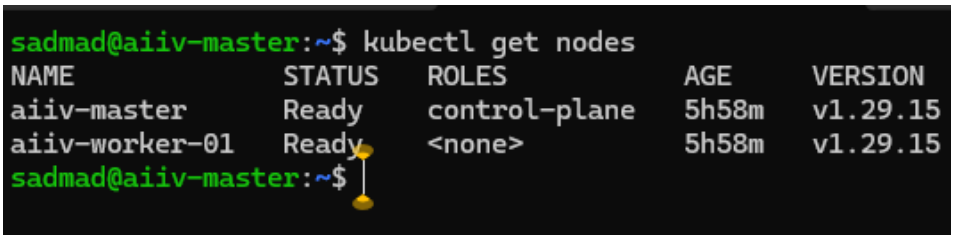

FIGURE 7.1 – Les nodes de cluster kubernetes

```
sadmad@aiiv-master:~$ kubectl get pods -n ai-interview
NAME                          READY   STATUS    RESTARTS        AGE
backend-65bd66499b-j47m7      1/1     Running   1 (3m33s ago)   5h27m
frontend-59f57bf8bf-gb56s     1/1     Running   1 (3m33s ago)   5h36m
postgres-69b466f5c5-f8xgx     1/1     Running   2 (3m33s ago)   5h27m
sadmad@aiiv-master:~$
```

FIGURE 7.2 – Les pods dans le cluster

```
sadmad@aiiv-master:~$ kubectl get svc -n ai-interview
NAME       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
backend    NodePort    10.103.124.255   <none>        8000:30800/TCP   5h46m
frontend   NodePort    10.104.41.225    <none>        80:30080/TCP     5h46m
postgres   ClusterIP   10.101.89.141    <none>        5432/TCP         5h46m
sadmad@aiiv-master:~$
```

FIGURE 7.3 – Les services de cluster

7.10 Flux de Déploiement Kubernetes

La figure 7.4 illustre l'ordre d'application des manifestes Kubernetes lors d'un déploiement via le pipeline Jenkins.

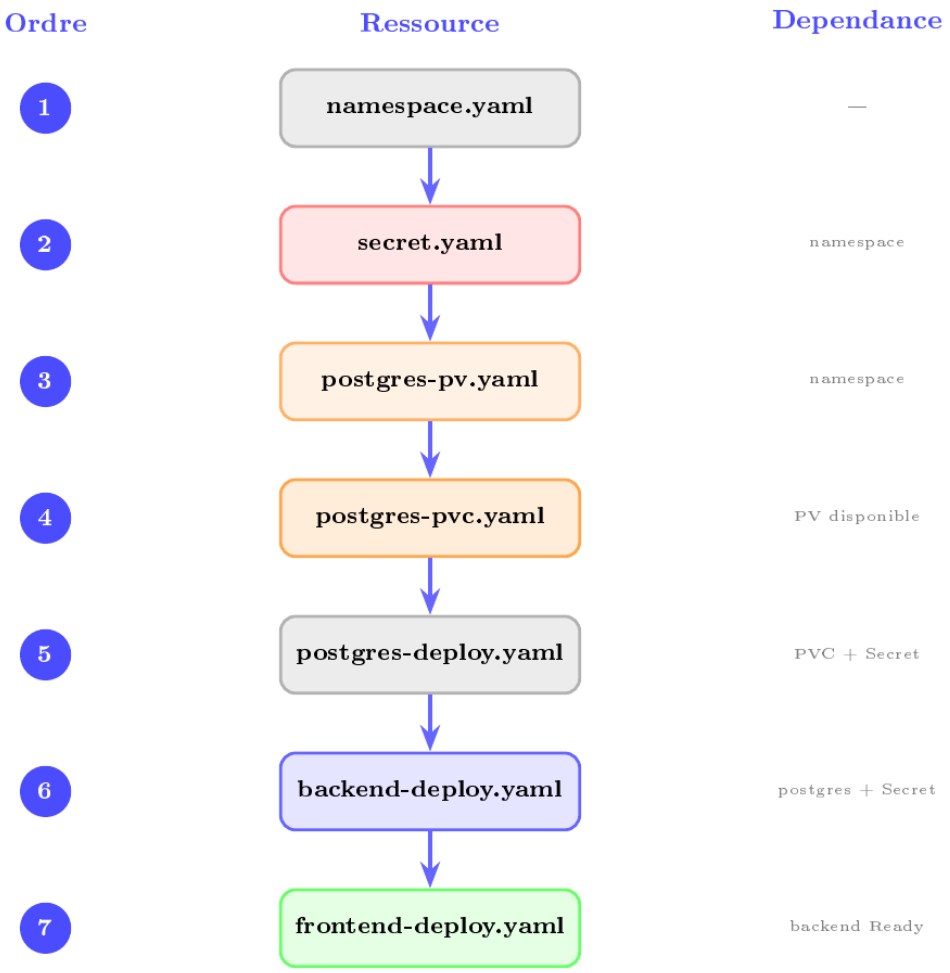

FIGURE 7.4 – Ordre d'application des manifestes Kubernetes lors du déploiement

CHAPITRE 8

Pipeline CI/CD avec Jenkins

8.1 Introduction

Jenkins est le serveur d'automatisation open-source utilisé dans ce projet pour orchestrer l'intégralité du cycle de vie de l'application. Le pipeline est défini sous forme de code dans un fichier `Jenkinsfile` versionné dans le dépôt GitHub, suivant le principe du *Pipeline as Code*.

Le pipeline est déclenché automatiquement à chaque `git push` via un **webhook GitHub**, et couvre deux grandes phases :

— **Phase CI** (Intégration Continue) : vérification de la qualité du code, exécution des tests, build du frontend et scan de sécurité.

— **Phase CD** (Déploiement Continu) : build des images Docker, test des conteneurs, et push vers Docker Hub.

8.2 Configuration Globale du Pipeline

Le pipeline est configuré avec les paramètres globaux suivants :

— **Agent** : `any` — le pipeline peut s'exécuter sur n'importe quel nœud Jenkins disponible.

— **Timeout** : 90 minutes — le pipeline est automatiquement annulé s'il dépasse cette durée.

— **Historique** : 10 builds conservés (`logRotator`).

— **Timestamps** : chaque ligne de log est horodatée pour faciliter le débogage.

8.2.1 Variables d'Environnement

Le pipeline définit les variables d'environnement suivantes, accessibles dans toutes les étapes :

TABLE 8.1 – Variables d'environnement du pipeline Jenkins

Variable	Valeur / Description
DOCKER_REGISTRY	othmansalahi (Docker Hub account)
DOCKER_IMAGE_BACKEND	othmansalahi/ai-voice-interview-backend
DOCKER_IMAGE_FRONTEND	othmansalahi/ai-voice-interview-frontend
DOCKER_TAG	Numéro de build Jenkins ($BUILD_NUMBER)
VENV_PATH	Chemin vers le virtualenv Python
DATABASE_URL	Injectée depuis Jenkins Credentials (sensible)
SECRET_KEY	Injectée depuis Jenkins Credentials (sensible)

8.3 Étapes du Pipeline

Le pipeline est composé de **11 étapes** (*stages*) exécutées séquentiellement. La figure 8.12 illustre le flux complet.

8.3.1 Stage 1 : Checkout

Jenkins récupère le code source depuis le dépôt GitHub via `checkout scm`. Le hash court du commit Git est extrait et stocké dans la variable `GIT_COMMIT_HASH`, qui sera utilisée comme tag Docker pour assurer la traçabilité entre l'image déployée et le code source exact.

```
1        // ────────────────────────────────────────
2        stage('Checkout') {
3        // ────────────────────────────────────────
4            steps {
5                echo 'Checking out source code...'
6                checkout scm
7                sh 'git rev-parse --short HEAD > .git/commit-hash'
8                script {
9                    env.GIT_COMMIT_HASH = readFile('.git/commit-hash').trim()
10                   echo "Commit: ${env.GIT_COMMIT_HASH}"
11               }
12           }
13       }
```

FIGURE 8.1 – stage checkout

8.3.2 Stage 2 : Check Docker Access

Cette étape vérifie si Docker est accessible directement ou via `sudo`. La commande Docker appropriée est stockée dans la variable `DOCKER_CMD` et réutilisée dans toutes les étapes suivantes. Cette approche permet de gérer les différentes configurations de permissions Docker selon l'environnement Jenkins.

```
// ────────────────────────────────────────
        stage('Check Docker Access') {
        // ────────────────────────────────────────
            steps {
                script {
                    if (sh(script: 'docker info > /dev/null 2>&1',
returnStatus: true) == 0) {
                        env.DOCKER_CMD = 'docker'
                        echo 'Docker accessible directly.'
                    } else if (sh(script:
'sudo docker info > /dev/null 2>&1', returnStatus: true) == 0) {
                        env.DOCKER_CMD = 'sudo docker'
                        echo 'Docker accessible via sudo.'
                    } else {
                        env.DOCKER_CMD = 'docker'
                        echo
'WARNING: Docker access could not be verified.'
                    }
                }
            }
        }
```

FIGURE 8.2 – stage docker access

8.3.3 Stage 3 : Setup Python Environment

Un environnement virtuel Python 3.11 isolé est créé dans le workspace Jenkins. Les dépendances sont installées en trois étapes :

1. Mise à jour de `pip`, `setuptools` et `wheel`.
2. Installation de **PyTorch 2.2.0** (version CPU uniquement) depuis le dépôt officiel PyTorch, compatible avec les VMs Azure sans GPU.
3. Installation des dépendances applicatives depuis `requirements.txt`.

```
        // ────────────────────────────────────────────
        stage('Setup Python Environment') {
        // ────────────────────────────────────────────
            steps {
                echo 'Setting up Python virtual environment...'
                dir('backend') {
                    sh '''
                        python3.11 -m venv ${VENV_PATH} --clear

                        ${VENV_PATH}/bin/pip install --upgrade pip setuptools wheel \
                            --no-cache-dir

                        ${VENV_PATH}/bin/pip install --no-cache-dir \
                            --index-url https://download.pytorch.org/whl/cpu \
                            --extra-index-url https://pypi.org/simple \
                            torch==2.2.0 torchvision==0.17.0

                        ${VENV_PATH}/bin/pip install --no-cache-dir -r requirements.txt
                    '''
                }
            }
        }
```

FIGURE 8.3 – stage setup python environment

8.3.4 Stage 4 : Setup Frontend Environment

Les dépendances Node.js du frontend sont installées via `npm ci` (au lieu de `npm install`) pour garantir une installation déterministe basée exactement sur le fichier `package-lock.json`, assurant la reproductibilité des builds.

```
1        // ──────────────────────────────────────────
2        stage('Setup Frontend Environment') {
3        // ──────────────────────────────────────────
4            steps {
5                echo 'Installing frontend dependencies...'
6                dir('frontend') {
7                    sh 'npm ci'
8                }
9            }
10       }
```

FIGURE 8.4 – stage setuup frontend environment

8.3.5 Stage 5 : Lint & Code Quality

Deux outils de qualité de code sont exécutés sur le backend :

— **Flake8** : analyse statique Python ciblant uniquement les erreurs critiques (`E9, F63, F7, F82`), notamment les erreurs de syntaxe, les imports non définis et les variables non utilisées.

— **Black** : vérification du formatage du code Python. Les deux outils sont configurés avec `|| true` pour ne pas bloquer le pipeline en cas d'avertissement non critique.

```
1        // ────────────────────────────────────────
2        stage('Lint & Code Quality') {
3        // ────────────────────────────────────────
4            steps {
5                echo 'Running linters and code quality checks...'
6                dir('backend') {
7                    sh '''
8                        ${VENV_PATH}/bin/pip install flake8 black --no-cache-dir
9
10                       echo "--- Flake8 (critical errors only) ---"
11                       ${VENV_PATH}/bin/flake8 . \
12                           --count \
13                           --select=E9,F63,F7,F82 \
14                           --show-source \
15                           --statistics || true
16
17                       echo "--- Black (format check) ---"
18                       ${VENV_PATH}/bin/black --check . || true
19                   '''
20               }
21           }
22       }
```

FIGURE 8.5 – stage Lint and code quality

8.3.6 Stage 6 : Run Frontend Tests

Les tests unitaires du frontend React sont exécutés via `npm run test`. L'option `|| true` permet de ne pas bloquer le pipeline si certains tests échouent, tout en conservant les résultats dans les logs Jenkins.

8.3.7 Stage 7 : Build Frontend

L'application React est compilée en fichiers statiques optimisés pour la production via `npm run build` (Vite). Les artefacts générés dans le répertoire `dist/` seront ensuite copiés dans l'image Docker du frontend lors de l'étape de build.

```
1        // ──────────────────────────────────────────
2        stage('Run Frontend Tests') {
3        // ──────────────────────────────────────────
4            steps {
5                echo 'Running frontend tests...'
6                dir('frontend') {
7                    sh 'npm run test || true'
8                }
9            }
10       }
11
12       // ──────────────────────────────────────────
13       stage('Build Frontend') {
14       // ──────────────────────────────────────────
15           steps {
16               echo 'Building frontend application...'
17               dir('frontend') {
18                   sh 'npm run build'
19               }
20           }
21       }
```

FIGURE 8.6 – stage test et build de frontend

8.3.8 Stage 8 : Run Backend Tests

Les tests unitaires Python sont exécutés avec **pytest**, en excluant les tests nécessitant une base de données (`-m "not database"`). Trois rapports sont générés automatiquement :

— **JUnit XML** (`test-results.xml`) : publié dans Jenkins pour afficher le tableau des tests et leur statut.

— **Coverage XML** : rapport de couverture en format XML.

— **Coverage HTML** (`htmlcov/index.html`) : rapport de couverture visuel publié comme artefact Jenkins accessible depuis l'interface web.

```
        // ────────────────────────────────────────────
        stage('Run Backend Tests') {
        // ────────────────────────────────────────────
            steps {
                echo 'Running backend test suite...'
                dir('backend') {
                    sh '''
                        ${VENV_PATH}/bin/pytest -v -m "not database" \
                            --junit-xml=test-results.xml \
                            --cov=. \
                            --cov-report=xml \
                            --cov-report=html \
                            --cov-report=term || true
                    '''
                }
            }
            post {
                always {
                    script {
                        if (fileExists('backend/test-results.xml')) {
                            junit 'backend/test-results.xml'
                            echo 'Test results published.'
                        } else {
                            echo 'No test results file found - skipping junit.'
                        }
                        if (fileExists('backend/htmlcov/index.html')) {
                            publishHTML(target: [
                                allowMissing         : true,
                                alwaysLinkToLastBuild: true,
                                keepAll              : true,
                                reportDir            : 'backend/htmlcov',
                                reportFiles          : 'index.html',
                                reportName           : 'Coverage Report'
                            ])
                            echo 'Coverage report published.'
                        } else {
                            echo
'No coverage report found - skipping HTML publish.'
                        }
                    }
                }
            }
        }
```

FIGURE 8.7 – stage test de backend

8.3.9 Stage 9 : Security Scan

L'outil **Safety** analyse les dépendances Python listées dans `requirements.txt` et les compare à une base de données de **CVE** (Common Vulnerabilities and Exposures) connues. Le rapport est généré en format JSON et archivé dans les logs Jenkins.

```
1        // ────────────────────────────────────────
2        stage('Security Scan') {
3        // ────────────────────────────────────────
4            steps {
5                echo 'Running security vulnerability scan...'
6                dir('backend') {
7                    sh '''
8                        ${VENV_PATH}/bin/pip install safety --no-cache-dir
9                        ${VENV_PATH}/bin/safety check --json || true
10                   '''
11               }
12           }
13       }
```

FIGURE 8.8 – stage security scan

8.3.10 Stage 10 : Build Docker Images

Les images Docker du backend et du frontend sont construites avec l'option `--no-cache` pour garantir un build propre. Chaque image reçoit **trois tags** simultanément :

— `:$BUILD_NUMBER` — numéro de build Jenkins pour le rollback.

— `:latest` — toujours pointé vers la dernière version.

— `:$GIT_COMMIT_HASH` — hash Git court pour la traçabilité du code.

Une particularité notable : le Dockerfile du backend est exécuté depuis la **racine du projet** (`-f ./backend/Dockerfile .`) pour donner accès au contexte global du projet si nécessaire.

```
1        // ────────────────────────────────────────
2        stage('Build Docker Images') {
3        // ────────────────────────────────────────
4            steps {
5                echo 'Building backend and frontend Docker images...'
6                sh '''
7                    echo "=== Building backend image from project root ==="
8                    ${DOCKER_CMD} build --no-cache \
9                        -f ./backend/Dockerfile \
10                       -t ${DOCKER_IMAGE_BACKEND}:${DOCKER_TAG} \
11                       -t ${DOCKER_IMAGE_BACKEND}:latest \
12                       -t ${DOCKER_IMAGE_BACKEND}:${GIT_COMMIT_HASH} \
13                       .
14
15                   echo "=== Building frontend image ==="
16                   ${DOCKER_CMD} build --no-cache \
17                       -t ${DOCKER_IMAGE_FRONTEND}:${DOCKER_TAG} \
18                       -t ${DOCKER_IMAGE_FRONTEND}:latest \
19                       -t ${DOCKER_IMAGE_FRONTEND}:${GIT_COMMIT_HASH} \
20                       ./frontend
21
22                   echo "=== Images built successfully ==="
23                   ${DOCKER_CMD} images | grep "ai-voice-interview"
24               '''
25           }
26       }
27
```

FIGURE 8.9 – stage build images de docker frontend et backend

8.3.11 Stage 11 : Test Docker Images

Avant le push, les images sont testées dans un environnement isolé :

1. Création d'un **réseau Docker dédié** au build (`interview-test-network-$BUILD_N`
2. Démarrage d'un conteneur **PostgreSQL 15** de test.
3. Attente de 30 secondes pour la disponibilité de la base de données.
4. Démarrage du conteneur **backend** connecté à la base de test.
5. Attente de **60 secondes** pour le chargement du modèle IA (Sentence-BERT et PyTorch nécessitent un temps d'initialisation).
6. Exécution d'un **health check** via `curl -f http://localhost:8001/health`.

Un bloc `post { always }` garantit le nettoyage complet des conteneurs et du réseau de test, même en cas d'échec.

```
1        // ────────────────────────────────────────────
2        stage('Test Docker Images') {
3        // ────────────────────────────────────────────
4            steps {
5                echo 'Testing backend Docker image health...'
6                script {
7                    sh """
8                        echo "=== Creating test network ==="
9                        ${DOCKER_CMD} network create \
10                           interview-test-network-${BUILD_NUMBER} 2>/dev/null || true
11
12                       echo "=== Starting test database ==="
13                       ${DOCKER_CMD} run -d \
14                       ...
15                       echo "=== Waiting for AI model to load (60s) ==="
16                       sleep 60
17
18                       echo "=== Running health check ==="
19                       curl -f http://localhost:8001/health || exit 1
20                       echo "=== Health check passed! ==="
21                   """
22               }
23           }
24           post {
25               always {
26                   sh """
27                       echo "=== Cleaning up test containers ==="
28                       ${DOCKER_CMD} stop  test-backend-${BUILD_NUMBER}  2>/dev/null || true
29                           ...
30                       ${DOCKER_CMD} network rm \
31                           interview-test-network-${BUILD_NUMBER} 2>/dev/null || true
32                       echo "=== Test cleanup done ==="
33                   """
34               }
35           }
36       }
```

FIGURE 8.10 – stage test docker images

8.3.12 Stage 12 : Push Docker Images

Les images sont poussées vers **Docker Hub** en utilisant les credentials Jenkins (`docker-credentials`) via le bloc `withCredentials`. Le mot de passe Docker n'apparaît jamais en clair dans les logs grâce à `-password-stdin`. Les six images (3 tags × 2 services) sont publiées.

```
        // ──────────────────────────────────────────
        stage('Push Docker Images') {
        // ──────────────────────────────────────────
            steps {
                echo 'Pushing images to Docker Hub...'
                withCredentials([usernamePassword(
                    credentialsId: 'docker-credentials',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh '''
                        echo "$DOCKER_PASS" | \
                            ${DOCKER_CMD} login -u "$DOCKER_USER" --password-stdin

                        echo "=== Pushing backend images ==="
                        ${DOCKER_CMD} push ${DOCKER_IMAGE_BACKEND}:${DOCKER_TAG}
                        ${DOCKER_CMD} push ${DOCKER_IMAGE_BACKEND}:latest
                        ${DOCKER_CMD} push ${DOCKER_IMAGE_BACKEND}:${GIT_COMMIT_HASH}

                        echo "=== Pushing frontend images ==="
                        ${DOCKER_CMD} push ${DOCKER_IMAGE_FRONTEND}:${DOCKER_TAG}
                        ${DOCKER_CMD} push ${DOCKER_IMAGE_FRONTEND}:latest
                        ${DOCKER_CMD} push ${DOCKER_IMAGE_FRONTEND}:${GIT_COMMIT_HASH}

                        ${DOCKER_CMD} logout
                        echo "=== All images pushed successfully! ==="
                    '''
                }
            }
        }
    }
```

FIGURE 8.11 – stage push images to docker hub

8.4 Bloc Post-Exécution

Le bloc `post` définit les actions exécutées selon le résultat du pipeline :

— **always** : nettoyage complet du workspace Jenkins (`cleanWs()`) pour libérer de l'espace disque.

— **success** : affichage d'un résumé détaillé des étapes réussies et des images disponibles sur Docker Hub.

— **failure** : affichage des causes d'échec les plus courantes pour faciliter le débogage (build Docker, tests, credentials, timeout).

8.5 Flux Complet du Pipeline

La figure 8.12 illustre le flux complet des 12 étapes du pipeline Jenkins avec les phases CI et CD clairement délimitées.

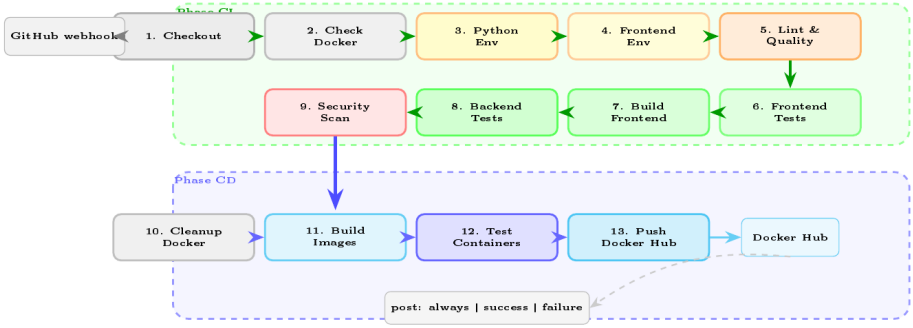

FIGURE 8.12 – Flux complet du pipeline CI/CD Jenkins — 13 stages

8.6 Gestion des Credentials Jenkins

Tous les secrets du pipeline sont gérés via le **magasin de credentials Jenkins**, garantissant qu'aucune information sensible n'apparaît dans le code source ou les logs. Le tableau 8.2 liste les credentials configurés.

TABLE 8.2 – Credentials Jenkins configurés pour le pipeline

ID du Credential	Type	Usage
`docker-credential`	Username/Passw	Authentification Docker Hub
`test-database-url`	Secret Text	URL de connexion PostgreSQL de test
`test-secret-key`	Secret Text	Clé secrète JWT pour les tests

8.7 Points Techniques Remarquables

Plusieurs décisions techniques notables ont été prises dans la conception du pipeline :

— **Isolation par BUILD_NUMBER** : les conteneurs et réseaux Docker de

test sont nommés avec le numéro de build (`test-backend-$BUILD_NUMBER`), permettant plusieurs builds concurrents sans conflit.

— **Attente du chargement IA** : une pause de 60 secondes est intentionnellement ajoutée après le démarrage du backend pour laisser le temps aux modèles Sentence-BERT et PyTorch de se charger en mémoire avant d'effectuer le health check.

— **Nettoyage préventif Docker** : une étape de `docker system prune` est exécutée avant le build des images pour éviter les problèmes d'espace disque sur le serveur Jenkins.

— **npm ci vs npm install** : l'utilisation de `npm ci` garantit une installation strictement conforme au `package-lock.json`, évitant les surprises liées aux mises à jour automatiques de dépendances.

— **PyTorch CPU uniquement** : PyTorch est installé depuis le dépôt officiel CPU (`download.pytorch.org/whl/cpu`) pour être compatible avec les VMs Azure standards sans GPU, réduisant également la taille du virtualenv.

CHAPITRE 9

Conclusion

9.1 Synthèse du Travail Réalisé

Ce projet avait pour objectif la mise en place d'une infrastructure **DevOps** complète pour l'application **AI Voice Interview**, en couvrant l'intégralité du cycle de vie logiciel, depuis le provisionnement des ressources cloud jusqu'au déploiement automatisé en production.

L'ensemble du travail s'est articulé autour de **trois couches complémentaires** qui forment une chaîne d'automatisation cohérente :

1. **Provisionnement cloud avec Terraform** : toutes les ressources Microsoft Azure ont été définies sous forme de code HCL et provisionnées de manière entièrement automatisée et reproductible. Les machines virtuelles Ubuntu 22.04, le réseau virtuel, les groupes de sécurité et les adresses IP sont créés à la demande par une simple exécution du pipeline Jenkins, avec une étape d'approbation manuelle protégeant contre toute modification accidentelle.
2. **Configuration des serveurs avec Ansible** : l'installation de Docker, l'initialisation du cluster Kubernetes via `kubeadm`, le déploiement du plugin réseau Flannel CNI et la récupération du *kubeconfig* sont entièrement automatisés par quatre playbooks Ansible enchaînés. L'inventaire est généré dynamiquement depuis les sorties Terraform, éliminant toute configuration manuelle.
3. **Orchestration Kubernetes** : l'application est déployée dans un namespace dédié avec des manifestes YAML définissant les Deployments, Services NodePort, PersistentVolumes et Secrets. La persistance des données PostgreSQL, la gestion sécurisée des credentials et le proxy nginx intégré au frontend constituent les points techniques les plus importants de cette couche.

Ces trois couches sont pilotées par **deux pipelines Jenkins** : un pipeline infrastructure déclenché manuellement, et un pipeline applicatif déclenché automatiquement à chaque `git push` via webhook GitHub. Ce dernier couvre 13

étapes allant du checkout du code jusqu'au push des images Docker sur Docker Hub, en passant par les tests, l'analyse de qualité, le scan de sécurité et le déploiement Kubernetes.

9.2 Bilan des Objectifs Atteints

Le tableau 9.1 présente le bilan des objectifs initiaux du projet et leur état de réalisation.

TABLE 9.1 – Bilan des objectifs du projet

Objectif	Statut	Outil utilisé
Application conteneurisée avec Dockerfile	Réalisé	Docker
Pipeline CI/CD automatisé	Réalisé	Jenkins
Cluster Kubernetes opérationnel	Réalisé	kubeadm + Flannel
Provisionnement IaC sur Azure	Réalisé	Terraform
Configuration automatique des serveurs	Réalisé	Ansible
Inventaire dynamique Ansible	Réalisé	Outputs Terraform
Sécurité : secrets, isolation réseau	Réalisé	K8s Secrets + NSG
Tests automatisés dans le pipeline	Réalisé	pytest + npm test
Scan de sécurité des dépendances	Réalisé	Safety (CVE)

9.3 Difficultés Rencontrées

La réalisation de ce projet a présenté plusieurs défis techniques :

— **Gestion des ressources Azure** : les tiers gratuits Azure imposent des limites strictes en termes de vCPU et de RAM disponibles par région. Le choix de la taille `Standard_B2s` a nécessité plusieurs ajustements pour équilibrer performance et contraintes budgétaires.

— **Chargement des modèles IA en mémoire** : les modèles Sentence-BERT et PyTorch nécessitent un temps d'initialisation significatif (environ 60 secondes) sur des machines sans GPU. Cela a imposé l'ajout d'une pause explicite dans le pipeline Jenkins avant l'exécution du health check.
— **Accès SSH aux workers** : l'absence d'IP publique sur les nœuds workers a nécessité la mise en place d'un mécanisme ProxyJump via le nœud maître, tant pour Ansible que pour les opérations kubectl depuis Jenkins.
— **Synchronisation Terraform et Ansible** : la génération dynamique de l'inventaire Ansible depuis les outputs Terraform a demandé une attention particulière pour gérer les cas de démarrage lent des machines virtuelles, résolue par un mécanisme de retry SSH avec 20 tentatives espacées de 15 secondes.
— **Espace disque sur Jenkins** : les builds Docker successifs sans nettoyage saturaient rapidement le disque du serveur CI-CD, ce qui a conduit à l'ajout d'une étape `docker system prune` systématique avant chaque build.

9.4 Perspectives d'Amélioration

Bien que les objectifs principaux aient été atteints, plusieurs axes d'amélioration peuvent être envisagés pour enrichir l'infrastructure :

— **Monitoring et Observabilité** : l'intégration de *Prometheus* pour la collecte des métriques Kubernetes et de *Grafana* pour leur visualisation permettrait de surveiller en temps réel la santé du cluster, l'utilisation des ressources et les performances de l'application.
— **Centralisation des logs** : le déploiement d'une stack *EFK* (Elasticsearch, Fluentd, Kibana) faciliterait la recherche et l'analyse des logs de tous les pods depuis une interface unifiée.
— **Mise à l'échelle automatique** : l'implémentation du *Horizontal Pod Autoscaler* (HPA) permettrait d'ajuster dynamiquement le nombre de

réplicas backend en fonction de la charge CPU ou du nombre de requêtes simultanées.

— **Ingress Controller et HTTPS** : le remplacement des Services NodePort par un *nginx Ingress Controller* avec certificat TLS (via *cert-manager* et Let's Encrypt) sécuriserait les communications et simplifierait le routage.

— **Approche GitOps** : la migration vers *ArgoCD* permettrait d'adopter un modèle de déploiement déclaratif basé sur Git, où l'état du cluster est automatiquement synchronisé avec le dépôt de manifestes Kubernetes.

— **Migration vers AKS** : le passage du cluster `kubeadm` auto-géré vers *Azure Kubernetes Service* (AKS) déléguerait la gestion du plan de contrôle à Azure, réduisant la charge opérationnelle et améliorant la disponibilité.

9.5 Conclusion Générale

Ce projet démontre qu'une infrastructure DevOps complète et fonctionnelle peut être mise en place à partir de zéro avec des outils open-source largement adoptés dans l'industrie. La combinaison **Terraform + Ansible + Kubernetes + Jenkins + Docker** constitue une chaîne d'outils cohérente et puissante, capable de gérer des applications complexes intégrant de l'intelligence artificielle.

Au-delà des aspects techniques, ce projet a permis de comprendre les enjeux réels d'une mise en production : la gestion des secrets, la sécurité réseau, la persistance des données, la reproductibilité des environnements et la traçabilité des déploiements. Ces compétences sont aujourd'hui indispensables pour tout ingénieur logiciel travaillant dans un contexte cloud moderne.

L'infrastructure réalisée constitue une base solide sur laquelle les améliorations futures (monitoring, autoscaling, GitOps) pourront être construites de manière incrémentale, sans remettre en cause l'architecture existante.

Références Bibliographiques

[1] N. Reimers and I. Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP*, pages 3982–3992. ACL, 2019.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT : Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, et al. PyTorch : An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 2019.

[4] G. Kim, J. Humble, P. Debois, and J. Willis. *The DevOps Handbook*. IT Revolution Press, 2016.

[5] J. Humble and D. Farley. *Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[6] Jenkins Project. Jenkins : The leading open source automation server. `https://www.jenkins.io/doc/`, 2024.

[7] HashiCorp. Terraform documentation. `https://developer.hashicorp.com/terraform/docs`, 2024.

[8] Y. Brikman. *Terraform : Up and Running*, 2nd edition. O'Reilly Media, 2019.

[9] Red Hat. Ansible documentation. `https://docs.ansible.com/`, 2024.

[10] Docker Inc. Docker documentation. `https://docs.docker.com/`, 2024.

[11] S. P. Kane and K. Matthias. *Docker : Up and Running*, 3rd edition. O'Reilly Media, 2023.

[12] The Kubernetes Authors. Kubernetes documentation. `https://kubernetes.io/docs/`, 2024.

[13] B. Burns, J. Beda, K. Hightower, and L. Evenson. *Kubernetes : Up and Running*, 3rd edition. O'Reilly Media, 2022.

[14] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14 :70–93, 2016.

[15] Microsoft. Microsoft Azure documentation. https://learn.microsoft.com/en-us/azure/, 2024.

[16] HashiCorp. Terraform AzureRM provider documentation. https://registry.terraform.io/providers/hashicorp/azurerm/latest/doc 2024.

[17] S. Ramirez. FastAPI : Modern, fast web framework for building APIs. https://fastapi.tiangolo.com/, 2024.

[18] Meta Platforms, Inc. React : A javascript library for building user interfaces. https://react.dev/, 2024.

[19] E. You and contributors. Vite : Next generation frontend tooling. https://vitejs.dev/, 2024.

[20] The PostgreSQL Global Development Group. PostgreSQL 15 documentation. https://www.postgresql.org/docs/15/, 2024.

[21] O. Salahi, A. Chabbaki, M. Houali, and H. Dellaji. AI Voice Interview : Infrastructure DevOps CI/CD. https://github.com/OthmanSALAHI/ai_voice_interview, 2026. Branche : sadmad.